Universidad Mariano Gálvez de Guatemala

Centro Universitario San Juan Sacatepéquez

Ingeniería en Sistemas de Información y Ciencias de la Computación

Cátedra: Arquitectura de Computadoras II

Catedrático: Ingeniero Wilmer Tezén

Informe Final

Luz Miriam Gil Aguilar	7590-23-2193
Cindy Beatriz Pú Getellá	7590-17-14351
Valeria Melissa Guzman Estrada	7590-23-4125
Angel Amaliel Mayén Cardona	7590-23-12475
Sindi Carina Acabal Hernández	7590-21-6522

Índice

Resumen 1

Sistema de monitoreo perimetral urbano para San Juan Sacatepéquez 2

- Objetivo general 2
- Objetivos específicos 2
- Cliente hipotético y alcance 3

Marco teórico 4

- Arquitectura orientada a eventos 4
- Correlación y procesamiento de eventos complejos 4
- Objetivos de nivel de servicio 4
- Privacidad de datos en ciudades inteligentes 4
- Registros de decisiones de arquitectura 5

Arquitectura propuesta 5

- Vista general 5
- Componentes 7
- Modelo de despliegue 7
- Distribución por nodos 8
- Segmentos de red 9
- Comunicación entre componentes 11
- Contrato de eventos y topics 11
- Almacenamiento 12
- Perfiles de despliegue 13
- Capacidad y recursos 13
- Crecimiento a varios servidores 14

Justificación técnica 15

Configuración técnica de servicios y contenedores 17
Organización del repositorio 17
Docker Compose 17
Imágenes propias 21
Variables de entorno 22
Configuración por componente 24
Puesta en marcha y operación 28
Verificación 28
Simulación de sensores urbanos 29
Herramientas de simulación 29
Organización de los scripts 30
Sectores simulados 30
Generación de eventos por sensor 31
Tasas y comportamiento en el tiempo 33
Eventos inválidos 34
Envío y reintentos 34
Modos de ejecución 36
Escenarios guionados 36
Reporte ciudadano manual 38
Placas buscadas y evaluación 38
Plan de carga con JMeter 38
Modelo de correlación e inferencia 40
Ubicación en la arquitectura 40
Flujo de procesamiento 40
Entrada: el evento estandarizado 42

Estado en Redis 42
Reglas de correlación 44
Inferencia: score de confianza 46
Control de repeticiones y duplicados 47
Detección y análisis de falsos positivos 48
Salida: la alerta 48
Objetivo de nivel de servicio (SLO) de latencia 48
Visualización en tiempo real 49
Configuración 49
Casos de uso cubiertos 50
Dashboard 1: Centro de Monitoreo San Juan Sacatepéquez 50
Dashboard 2: Operación y SLOs 52
Dashboard 3: Histórico 55
Dashboard 4: Bandeja de instituciones 57
Dashboard 5: Despacho y falsos positivos 59
Dashboard 6: Recorrido de un pánico 61
Evaluación de resultados 63
Correlación 63
Latencia y capacidad 65
Resiliencia 67
Calidad de datos y SLOs 67
Comparación con casos reales de ciudades inteligentes 68
Centro de Operaciones de Río de Janeiro 68
Barcelona 68
Detección acústica de disparos en St. Louis 68

Limitaciones y trabajo futuro 70
Conclusiones 71
Referencias 72
Anexo A. Registros de decisiones de arquitectura (ADRs) 74
ADR-01. Plataforma de despliegue 74
ADR-02. Broker de eventos 77
ADR-03. Persistencia de eventos 78
ADR-04. Orquestación de tareas periódicas 80
ADR-05. Contenerización y orquestación 82
ADR-06. Resiliencia y tolerancia a fallos 83
ADR-07. Seguridad y privacidad de datos 85
ADR-08. Observabilidad y SLOs 87
ADR-09. Diseño de topics y clave de partición 89
ADR-10. Modelo de correlación e inferencia 90
ADR-11. Lenguaje de implementación 92
ADR-12. Formato de eventos 93

Resumen

Este informe presenta el diseño, la justificación y la evaluación de una simulación funcional de un sistema de monitoreo perimetral urbano para la Municipalidad de San Juan Sacatepéquez, Guatemala. El sistema recibe eventos de cinco tipos de sensores simulados (botones de pánico, cámaras lectoras de placas, radares de velocidad, sensores acústicos y reportes ciudadanos), los valida, los transporta mediante Apache Kafka, los correlaciona con siete reglas de negocio que mantienen su estado en Redis, persiste eventos y alertas en PostgreSQL, avisa a la PNC, Bomberos, CONRED y una aseguradora, y los visualiza en Grafana. La arquitectura es orientada a eventos y distribuida a nivel lógico, desplegada on-premise en un solo servidor con Docker Compose, decisión justificada por el presupuesto limitado del cliente y la sensibilidad de los datos. Las decisiones se documentaron en doce Architecture Decision Records. La evaluación mostró que los nueve escenarios guionados generaron su alerta esperada, con latencias de 30 a 200 ms en tráfico normal; que el sistema sostiene unos 510 eventos por segundo cumpliendo el objetivo de latencia de 2 s, alrededor de 100 veces el tráfico normal estimado; y que la caída de un ingestor, del correlador o de Redis no provocó pérdida de eventos ni alertas duplicadas.

Sistema de monitoreo perimetral urbano para San Juan Sacatepéquez

Las ciudades generan cada día una gran cantidad de señales sobre lo que ocurre en sus calles: llamadas de emergencia, cámaras de tránsito, reportes de vecinos y alarmas. El valor de un centro de monitoreo no está en recibir esas señales por separado, sino en relacionarlas a tiempo para reconocer un incidente real y avisar a quien puede atenderlo. Este proyecto diseña y construye una simulación funcional de ese tipo de sistema para un cliente concreto, la Municipalidad de San Juan Sacatepéquez, y actúa como un equipo de arquitectura de software: cada decisión técnica relevante se respalda con un análisis de alternativas y una justificación comprensible para actores no técnicos.

Objetivo general

Diseñar, justificar y desarrollar una simulación funcional de un sistema de monitoreo perimetral urbano con capacidades de detección, procesamiento, correlación y visualización de eventos críticos, basado en una arquitectura distribuida.

Objetivos específicos

1. Simular el comportamiento, la frecuencia, la ubicación y el tipo de eventos de cinco clases de sensores urbanos, incluidos picos de carga y eventos inválidos.
2. Implementar una arquitectura orientada a eventos con ingesta balanceada, transporte en Kafka, correlación con estado en memoria, persistencia y avisos a instituciones externas.
3. Correlacionar múltiples fuentes para inferir incidentes como un posible robo en curso o un accidente vial, y distinguir los falsos positivos.
4. Visualizar en tiempo real mapas de calor, listas de incidentes, alertas críticas e indicadores de servicio.
5. Medir la latencia, la capacidad y el comportamiento ante fallas, y documentar las decisiones de arquitectura en ADRs.

Cliente hipotético y alcance

El cliente es la Municipalidad de San Juan Sacatepéquez, del departamento de Guatemala, con 218 156 habitantes (Instituto Nacional de Estadística [INE], 2019). Es un municipio mediano, con presupuesto limitado y sin un equipo grande de tecnología, que necesita un centro de monitoreo propio. El sistema cubre siete sectores con coordenadas reales: Casco urbano, Sajcavillá, Cruz Blanca, Lo de Mejía, Ciudad Quetzal, Villas del Quetzal y Montúfar. Tres de ellos tienen zona escolar y cada uno tiene su propio límite de velocidad. No se usan sensores físicos: los sensores se emulan con programas en Python y con JMeter.

Marco teórico

Arquitectura orientada a eventos

En una arquitectura orientada a eventos, los componentes no se llaman directamente entre sí: publican hechos (eventos) en un intermediario y otros componentes los consumen cuando les corresponde. Esto desacopla a quien produce de quien consume, permite que varios consumidores reaccionen al mismo hecho y absorbe los picos de carga, porque los eventos esperan en el intermediario si un consumidor va más lento. Apache Kafka implementa esta idea como un registro de mensajes distribuido, persistente y ordenado por partición, diseñado originalmente para procesar grandes volúmenes de registros de actividad.

Correlación y procesamiento de eventos complejos

El procesamiento de eventos complejos busca patrones entre eventos simples para inferir situaciones de mayor nivel, normalmente dentro de ventanas de tiempo. En este proyecto, un pánico, una cámara que ve un vehículo saliendo y un radar que mide exceso de velocidad en el mismo sector y dentro de una ventana de tiempo permiten inferir un posible robo en curso, aunque ninguno de esos eventos lo indique por sí solo.

Objetivos de nivel de servicio

Un objetivo de nivel de servicio (SLO) es un umbral medible de lo que el sistema promete, junto con lo que ocurre cuando no se cumple (Beyer et al., 2016). Para un sistema de alertas, la métrica más importante es la latencia entre el evento y la alerta: una alerta de robo pierde valor si llega minutos después.

Privacidad de datos en ciudades inteligentes

Las placas vehiculares y las ubicaciones son datos personales. La seudonimización reemplaza un dato identificable por un valor que no permite recuperarlo sin información adicional guardada aparte. Un código de autenticación de mensajes con clave, como HMAC-SHA256 (Krawczyk et al., 1997), permite saber si dos lecturas corresponden a la misma placa sin conocer la placa. En Guatemala, la Ley de Acceso a la Información Pública reconoce la protección de los datos personales en poder de las instituciones públicas (Congreso de la República de Guatemala, 2008).

Registros de decisiones de arquitectura

Un Architecture Decision Record (ADR) es un documento breve que registra el contexto de una decisión, las alternativas, la elección y sus consecuencias (Nygard, 2011). El proyecto documenta doce ADRs, reunidos en el Anexo A.

Arquitectura propuesta

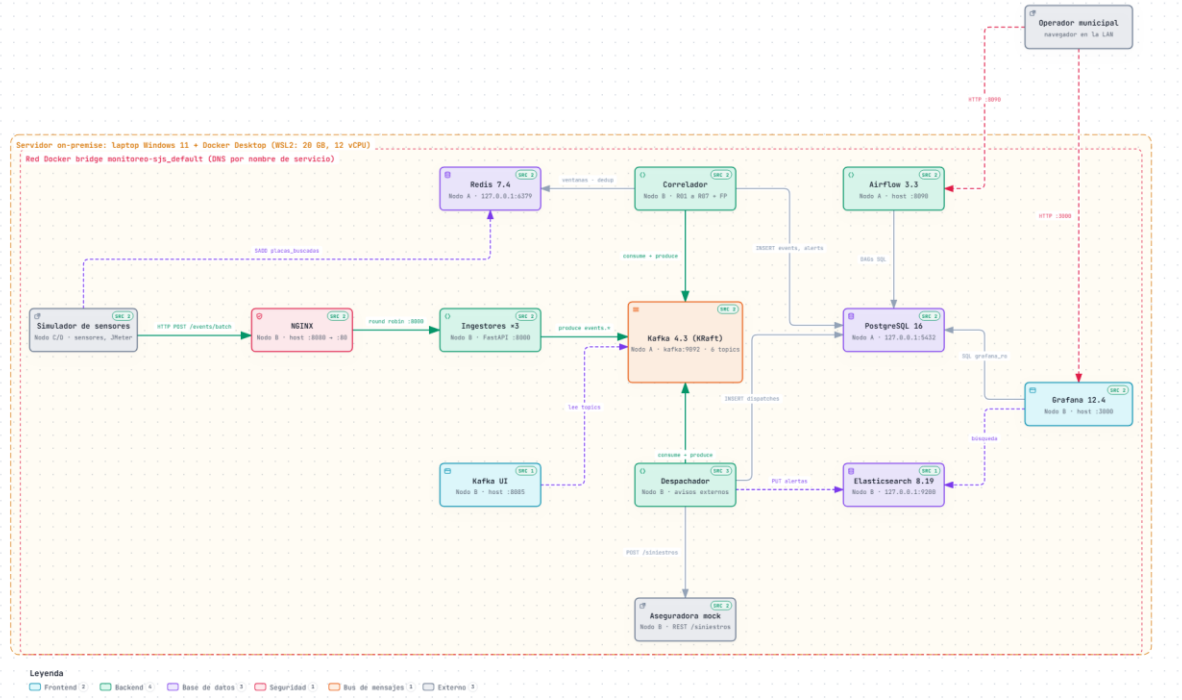
Vista general

La Figura 1 muestra la infraestructura: nodos, servicios, red y puertos. El flujo principal es el siguiente. Los sensores simulados envían eventos en JSON por HTTP a NGINX, que los reparte en turnos entre tres ingestores. Cada ingestor valida el evento, rechaza los inválidos hacia una cola de mensajes muertos y publica los válidos en Kafka con el sector como clave. El correlador consume esos eventos, consulta y actualiza su estado en Redis, aplica las reglas, guarda eventos y alertas en PostgreSQL y publica las alertas en otro topic de Kafka. El despachador lee las alertas y las envía a la institución que corresponde, por Kafka o por REST, y las indexa en Elasticsearch. Grafana muestra todo en tiempo real y Airflow ejecuta las tareas periódicas.

La Figura 1 es el esquemático de la infraestructura. Cada caja es un contenedor e indica a qué nodo lógico pertenece y por qué puerto se accede. Las dos fronteras punteadas representan el servidor físico y la red interna de Docker; el operador municipal queda fuera de ambas. Las flechas indican quién abre la conexión (cliente hacia servidor).

Figura 1

Infraestructura del sistema: nodos, servicios y red



Componentes

Tabla 1

Componentes del sistema

Componente	Tecnología	Función	Nodo
Simulador	Python, JMeter	Genera eventos de los cinco sensores, picos de carga, escenarios guionados y reportes manuales	C/D
Balanceador	NGINX 1.30	Única puerta de entrada (puerto 8080); reparte en turnos y tolera la caída de un ingestor	B
Ingestores (×3)	Python, FastAPI, Pydantic	Validan, estandarizan y publican en Kafka; responden 503 si Kafka no confirma	B
Broker	Apache Kafka 4.3 (KRaft)	Transporta y conserva los eventos 7 días en seis topics	A
Correlador	Python	Aplica las reglas R01 a R07 y detecta falsos positivos	B
Estado	Redis 7.4 (AOF)	Ventanas por sector, vehículos, duplicados y cooldowns	A
Base de datos	PostgreSQL 16	Eventos, alertas, envíos, inválidos, violaciones de SLO y estadísticas	A
Despachador	Python	Enruta alertas a PNC, Bomberos, CONRED y aseguradora; indexa en Elasticsearch	B
Aseguradora	Python, FastAPI	Entidad externa simulada que recibe accidentes por REST	B
Búsqueda	Elasticsearch 8.19	Búsqueda histórica de alertas	B
Visualización	Grafana 12.4	Seis dashboards: operador, SLOs, histórico, instituciones, despacho y falsos positivos, y recorrido de un pánico	B
Tareas periódicas	Apache Airflow 3.3	Retención de datos, reporte diario y análisis de falsos positivos	A

Modelo de despliegue

El sistema se despliega on-premise en un solo servidor propio de la municipalidad. Todos los servicios corren en contenedores Docker (Merkel, 2014) y se

comunican entre sí únicamente por red, por lo que la arquitectura es distribuida a nivel lógico, aunque comparta máquina. Esta elección responde al cliente: un municipio mediano con presupuesto limitado, que maneja datos personales (placas y ubicaciones) y que no debe depender del internet para recibir y procesar alertas.

Tabla 2

Servidor físico del prototipo

Elemento	Valor
Equipo	Laptop Acer Predator (servidor del prototipo)
Procesador	Intel Core i7-13650HX, 20 hilos
Memoria	32 GB de RAM (20 GB asignados a Docker)
Sistema operativo	Windows 11 con Docker Desktop sobre WSL2 (núcleo Linux)
Recursos para contenedores	20 GB de RAM, 12 núcleos y 4 GB de intercambio (archivo .wslconfig)
Imágenes de contenedores	Linux (Debian y Alpine), con versiones fijas
Recomendado para producción	Servidor de 8 a 16 núcleos, 32 a 64 GB de RAM, discos en espejo, UPS y Ubuntu Server 22.04 LTS

Distribución por nodos

La sección 6 del enunciado del proyecto propone una distribución de referencia en nodos. El proyecto la respeta como separación lógica y la despliega en un solo servidor físico (Tabla 3) y el esquemático de la Figura 1 la muestra gráficamente. Gracias a esa separación, si la municipalidad adquiere más equipos, cada nodo puede pasar a su propia máquina sin cambiar el código.

Tabla 3

Distribución de servicios por nodo

Nodo	Función primaria (enunciado del proyecto)	Servicios del proyecto	SO recomendado
A	Kafka, Airflow, PostgreSQL, Redis	kafka, kafka-init, postgres, redis, airflow	Ubuntu Server 22.04
B	Microservicios, Elastic, Grafana	nginx, ingestor-1, ingestor-2, ingestor-3, correlador, despachador, aseguradora-mock, elasticsearch, grafana, kafka-ui	Ubuntu Server 22.04 (enunciado: macOS o Linux)
C/D	Generación de eventos (clientes)	simulador (scripts Python) y JMeter	Windows o Linux (equipos de los alumnos)

Nota. En el prototipo los tres nodos comparten el mismo servidor físico (Windows 11 + Docker Desktop con WSL2), decisión registrada en el ADR-01.

Segmentos de red

El diseño distingue tres segmentos de red con distinto nivel de exposición, de acuerdo con el principio de exponer solo lo necesario (Tabla 4).

Tabla 4

Segmentos de red

Segmento	Quién se conecta	Qué se expone
1. Red local de la municipalidad (LAN)	Operadores con navegador y sensores o simuladores externos	Solo los puertos de uso humano y de entrada de eventos: 8080, 3000, 8085 y 8090
2. Interfaz local del servidor (127.0.0.1)	Solo el propio servidor (administración y depuración)	Bases de datos y broker: 5432, 6379, 9094 y 9200
3. Red interna de Docker (bridge monitoreo-sjs_default)	Los contenedores entre sí, por nombre de servicio (DNS interno de Docker)	Todos los puertos internos; los ingestores, el correlador, el despachador y la aseguradora no publican ningún puerto fuera de esta red

Tabla 5

Puertos publicados en el servidor

Puerto en el servidor	Servicio (puerto interno)	Segmento	Uso
8080	nginx (80)	LAN	Entrada de eventos: POST /events y /events/batch
3000	grafana (3000)	LAN	Dashboards para el operador
8085	kafka-ui (8080)	LAN	Ver topics y mensajes
8090	airflow (8080)	LAN	Interfaz de tareas periódicas
5432	postgres (5432)	127.0.0.1	Administración de la base de datos
6379	redis (6379)	127.0.0.1	Depuración del estado del correlador
9094	kafka (9094)	127.0.0.1	Clientes de Kafka desde el servidor
9200	elasticsearch (9200)	127.0.0.1	Consultas directas al índice de alertas

Comunicación entre componentes

Tabla 6

Conexiones, protocolos y puertos internos

Origen	Destino	Protocolo y puerto	Propósito
Simulador / JMeter	nginx	HTTP :80 (8080 en el servidor)	Enviar lotes de eventos
nginx	ingestor-1/2/3	HTTP :8000	Reparto en turnos (round robin)
Ingestores	kafka	Kafka :9092	Publicar eventos válidos e inválidos
Correlador	kafka	Kafka :9092	Leer eventos y publicar alertas
Correlador	redis	RESP :6379	Ventanas, vehículos y duplicados
Correlador	postgres	PostgreSQL :5432	Guardar eventos, alertas y violaciones de SLO
Despachador	kafka	Kafka :9092	Leer alertas e inválidos; publicar avisos a instituciones
Despachador	aseguradora-mock	HTTP :8000	POST /siniestros
Despachador	elasticsearch	HTTP :9200	Indexar alertas
Despachador	postgres	PostgreSQL :5432	Registrar envíos e inválidos
Grafana	postgres / elasticsearch	PostgreSQL :5432 / HTTP :9200	Consultas de los dashboards
Airflow	postgres	PostgreSQL :5432	Tareas periódicas
kafka-ui	kafka	Kafka :9092	Mostrar topics y mensajes

Contrato de eventos y topics

Todos los eventos comparten un sobre con los bloques device, location, data y metadata, un identificador único y dos marcas de tiempo: cuándo ocurrió el evento y cuándo se envió. Se usan seis topics organizados por etapa del flujo (Tabla 7). Se consideró usar un topic por tipo de sensor, pero se descartó porque el correlador necesita recibir en orden, y por un solo canal, los eventos de distintos tipos de un mismo sector; la separación se hizo por etapa y por consumidor (ADR-09).

Tabla 7

Topics de Kafka

Topic	Particiones	Productor	Consumidor
events.standardized	3 (clave = sector)	Ingestores	Correlador
events.invalid	1	Ingestores	Despachador (tabla invalid_events)
correlated.alerts	3	Correlador	Despachador
external.pnc	1	Despachador	PNC
external.bomberos	1	Despachador	Bomberos
external.conred	1	Despachador	CONRED

Nota. Retención de 7 días; la creación automática de topics está desactivada y los crea el contenedor kafka-init (Kreps et al., 2011).

Almacenamiento

Tabla 8

Volúmenes persistentes

Volumen	Servicio	Contenido
kafka-datos	kafka	Registro de mensajes de los topics (7 días)
postgres-datos	postgres	Eventos, alertas, envíos, estadísticas y auditoría
redis-datos	redis	Archivo AOF con el estado del correlador
elasticsearch-datos	elasticsearch	Índice de búsqueda de alertas
grafana-datos	grafana	Estado interno de Grafana
airflow-datos	airflow	Metadatos (SQLite) y registros de las tareas

Nota. Los volúmenes sobreviven a la recreación de los contenedores, solo se borran con docker compose down -v.

Perfiles de despliegue

Tabla 9

Perfiles de Docker Compose

Perfil	Servicios	Uso
core	kafka, kafka-init, kafka-ui, postgres, redis, ingestores, nginx, correlador, despachador, aseguradora-mock y grafana	Desarrollo y pruebas rápidas
full	Todo lo de core más elasticsearch y airflow (el despachador se reemplaza por despachador-es)	Pruebas finales y demostración
simulador	simulador (se ejecuta a demanda y termina)	Generación de eventos

Capacidad y recursos

Cada contenedor tiene un límite de memoria para que un servicio descontrolado no afecte a los demás. La suma de los límites del perfil completo es de unos 12 GB, dentro de los 20 GB asignados. Con la carga máxima sostenida medida (508 eventos por segundo), el sistema completo usó unos 4.6 GB de memoria y menos de un núcleo de los doce disponibles, el detalle de las pruebas está en el capítulo Evaluación de resultados.

Tabla 10

Límites de memoria por servicio

Servicio	Límite	Uso medido bajo carga
airflow	3 GB	1.34 GB
elasticsearch	2 GB	1.6 GB
kafka	1.5 GB	0.45 GB
postgres	1 GB	0.24 GB
kafka-ui	768 MB	336 MB
correlador, redis, kafka-init, simulador, grafana	512 MB c/u	de 24 a 182 MB
ingestor-1/2/3	384 MB c/u	de 40 a 54 MB
despachador	256 MB	24 MB
nginx, aseguradora-mock	128 MB c/u	28 y 48 MB

Crecimiento a varios servidores

Si la municipalidad incorpora más equipos, el diseño permite repartir los nodos sin modificar el código:

1. Instalar Docker en cada servidor y copiar el repositorio.
2. Llevar los servicios del nodo A a un servidor, los del nodo B a otro y los generadores a los equipos de los clientes (nodos C y D).
3. Cambiar los nombres de servicio por las direcciones IP o nombres de red correspondientes en las variables de entorno (por ejemplo KAFKA_BOOTSTRAP, POSTGRES_HOST y REDIS_HOST) y anunciar el listener externo de Kafka con la dirección del servidor A.
4. Abrir en el firewall solo los puertos necesarios entre servidores (9092, 5432, 6379, 9200 y 8000) y mantener en la LAN únicamente 8080, 3000, 8085 y 8090.
5. Con tres servidores o más, convertir Kafka en un clúster de tres brokers con factor de replicación 3 para eliminar el punto único de falla del broker.

Justificación técnica

Las decisiones de arquitectura se resumen en la Tabla 11 y se desarrollan en el Anexo A, cada una con su contexto, alternativas, consecuencias, impacto en el negocio y evidencia medida.

Tabla 11

Resumen de las decisiones de arquitectura

ADR	Decisión	Razón principal
ADR-01	On-premise en un solo servidor	Costo bajo y fijo (inversión estimada de Q28 000 a Q45 000 frente a unos Q84 000 a Q120 000 al año en la nube), datos bajo control municipal y latencia sin depender del internet
ADR-02	Apache Kafka	Conserva y ordena los eventos; si un consumidor se cae, no se pierden
ADR-03	PostgreSQL, Redis y Elasticsearch	Relacional para auditoría; memoria rápida para correlacionar; búsqueda solo para el histórico
ADR-04	Airflow standalone	Historial y ejecución manual de las tareas, aunque a esta escala no se justifica por carga
ADR-05	Docker Compose sin Kubernetes	En un solo servidor Kubernetes no aporta alta disponibilidad y sí complejidad
ADR-06	Al menos una vez + deduplicación	Cero pérdidas ante caídas de componentes distintos de Kafka
ADR-07	HMAC de placas y retención	Una fuga no revela placas; datos borrados a los 30 días
ADR-08	SLOs: p95 < 2 s, < 2 % inválidos	Incumplimientos registrados y visibles, no solo dashboards
ADR-09	Topics por etapa, clave = sector	Orden por sector para la correlación
ADR-10	Reglas determinísticas	Explicables y auditables ante la dirección y el área legal
ADR-11	Solo Python	Todo el código es fácilmente explicable
ADR-12	JSON con schemaVersion	Legible y evolucionable sin servidores adicionales

Un punto que suele cuestionarse es por qué se considera distribuido un sistema que corre en una sola máquina. La respuesta es que cada componente es un proceso independiente, en su propio contenedor, que solo se comunica con los demás por la

red (HTTP, Kafka, protocolos de PostgreSQL y Redis). Ninguno comparte memoria ni archivos con otro. Si la municipalidad compra un segundo servidor, basta mover contenedores y cambiar direcciones de red, sin modificar el código. La decisión de no usar Kubernetes es consciente: en un solo nodo agrega un plano de control y muchos conceptos sin dar alta disponibilidad real.

Configuración técnica de servicios y contenedores

Organización del repositorio

Tabla 12

Estructura del repositorio

Carpeta o archivo	Contenido
docker-compose.yml	Definición de todos los servicios, perfiles, redes y volúmenes
.env.example	Plantilla de variables de entorno (el archivo .env)
config/sectores.json	Los siete sectores del municipio: coordenadas, peso, radio, zona escolar y límite de velocidad
infra/nginx/nginx.conf	Balanceador de carga
infra/kafka/crear-topics.sh	Creación de los seis topics
infra/postgres/init/	Esquema, usuarios y carga de sectores de PostgreSQL
infra/grafana/	Datasources, proveedor de dashboards y los seis dashboards en JSON
servicios/ingestor, correlador, despachador, aseguradora_mock	Código, Dockerfile y requirements.txt de cada servicio propio
simulador/	Scripts de simulación de eventos
airflow/dags/	Tareas periódicas
pruebas/	Plan de carga de JMeter

Docker Compose

Un solo archivo docker-compose.yml define el sistema completo. Se aplicaron las mismas reglas a todos los servicios: versiones fijas de imagen (nunca latest), límite de memoria (mem_limit), reinicio automático (restart: unless-stopped) en los servicios de larga duración, chequeos de salud (healthcheck) en los servicios de datos y dependencias por estado sano (depends_on con condition: service_healthy), de modo que ningún servicio arranca antes que aquello que necesita. Los servicios se agrupan

en perfiles: core (desarrollo), full (agrega Elasticsearch y Airflow) y simulador (a demanda).

Tabla 13

Configuración de cada servicio

Servicio	Imagen	Perfil	Memoria	Puertos	Depende de
kafka	apache/kafka:4.3.1	core, full	1.5 GB	127.0.0.1:9094	-
kafka-init	apache/kafka:4.3.1	core, full	512 MB	-	kafka (sano)
kafka-ui	kafbat/kafka-ui:v1.5.0	core, full	768 MB	8085→8080	kafka (sano)
postgres	postgres:16.15	core, full	1 GB	127.0.0.1:5432	-
redis	redis:7.4.11	core, full	512 MB	127.0.0.1:6379	-
ingestor-1/2/3	monitoreo-sjs/ingestor:1.0	core, full	384 MB	-	kafka (sano), kafka-init (terminado)
nginx	nginx:1.30.5-alpine	core, full	128 MB	8080→80	ingestores
correlador	monitoreo-sjs/correlador:1.0	core, full	512 MB	-	kafka-init, postgres, redis
despachador	monitoreo-sjs/despachador:1.0	core	256 MB	-	kafka-init, postgres, aseguradora
despachador-es	monitoreo-sjs/despachador:1.0	full	256 MB	-	lo mismo + elasticsearch (sano)
aseguradora-mock	monitoreo-sjs/aseguradora-mock:1.0	core, full	128 MB	-	-
grafana	grafana/grafana:12.4.12	core, full	512 MB	3000→3000	postgres (sano)
elasticsearch	elasticsearch:8.19.22	full	2 GB	127.0.0.1:9200	-
airflow	apache/airflow:3.3.2-python3.12	full	3 GB	8090→8080	postgres (sano)
simulador	monitoreo-sjs/simulador:1.0	simulador	512 MB	-	-

Nota. Todos los servicios de larga duración usan restart: unless-stopped; kafka-init usa restart: no porque termina después de crear los topics.

Los tres ingestores comparten la misma definición mediante un ancla de YAML (&ingestor) y solo cambian su identificador (Figura 2). Ninguno publica puertos: solo se llega a ellos a través de NGINX.

Figura 2

Definición del ingestor en docker-compose.yml

```
# Tres copias del mismo código; solo cambia INGESTOR_ID.
  # No publican puertos: solo se llega a ellos a través de NGINX.
  ingestor-1: &ingestor
    build: ./servicios/ingestor
    image: monitoreo-sjs/ingestor:1.0
    profiles: ["core", "full"]
    restart: unless-stopped
    mem_limit: 384m
    environment:
      INGESTOR_ID: ingestor-1
      KAFKA_BOOTSTRAP: kafka:9092
      SECTORES_PATH: /config/sectores.json
      TZ: ${TZ}
    volumes:
      - ./config:/config:ro
    depends_on:
      kafka:
        condition: service_healthy
      kafka-init:
        condition: service_completed_successfully
```

Nota. Fragmento del archivo docker-compose.yml, líneas 128 a 147.

Tabla 14

Chequeos de salud

Servicio	Comando	Intervalo	Reintentos
kafka	kafka-broker-api-versions.sh --bootstrap-server localhost:9092	10 s	12 (espera inicial 20 s)
postgres	pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}	5 s	12
redis	redis-cli ping	5 s	12
elasticsearch	curl a /_cluster/health y espera estado green o yellow	10 s	30 (espera inicial 30 s)

Imágenes propias

Los cinco servicios propios (ingestor, correlador, despachador, aseguradora simulada y simulador) se construyen a partir de python:3.12.15-slim con un Dockerfile corto. Las dependencias se instalan antes de copiar el código para que Docker reutilice esa capa cuando solo cambia el código (Figura 3). Cada servicio tiene su propio requirements.txt con versiones fijas (Tabla 15).

Figura 3

Dockerfile del ingestor

```
# Imagen del ingestor: Python 3.12 + FastAPI servido con uvicorn.
FROM python:3.12.15-slim

WORKDIR /app
ENV PYTHONUNBUFFERED=1

# Primero las dependencias: Docker reutiliza esta capa si solo cambia el código
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY *.py .

EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000", "--no-access-log"]
```

Nota. Fragmento del archivo servicios/ingestor/Dockerfile, líneas 1 a 14.

Tabla 15

Dependencias de Python por servicio

Servicio	Librerías y versiones
ingestor	fastapi 0.142.2, uvicorn 0.54.0, pydantic 2.13.5, confluent-kafka 2.15.1
correlador	confluent-kafka 2.15.1, redis 7.4.1, psycopg2-binary 2.9.13
despachador	confluent-kafka 2.15.1, psycopg2-binary 2.9.13, requests 2.34.2
aseguradora-mock	fastapi 0.142.2, uvicorn 0.54.0
simulador	requests 2.34.2, redis 7.4.1, psycopg2-binary 2.9.13

Variables de entorno

Las contraseñas, la clave de las placas y los parámetros del correlador se leen del archivo .env, que no se sube al repositorio. Se entrega .env.example como plantilla;

el manual de instalación (README) incluye un comando que crea el .env con contraseñas aleatorias.

Tabla 16

Variables de entorno del sistema

Variable	Valor de ejemplo	Uso
TZ	America/Guatemala	Zona horaria de los contenedores
POSTGRES_DB / POSTGRES_USER / POSTGRES_PASSWORD	monitoreo / admin / (secreto)	Base de datos y usuario administrador
APP_DB_USER / APP_DB_PASSWORD	app / (secreto)	Usuario de lectura y escritura de los servicios
GRAFANA_RO_PASSWORD	(secreto)	Usuario grafana_ro, solo lectura
GRAFANA_ADMIN_USER / GRAFANA_ADMIN_PASSWORD	admin / (secreto)	Acceso a la interfaz de Grafana
AIRFLOW_ADMIN_PASSWORD	(secreto)	Acceso a la interfaz de Airflow
PLATE_HMAC_KEY	(64 caracteres hexadecimales)	Clave del HMAC de las placas
CORRELATION_WINDOW_SECONDS	300 (60 en la demostración)	Ventana de correlación
ACOUSTIC_MIN_PROBABILITY	0.7	Umbral de la regla de ataque armado
SLO_LATENCIA_MS	2000	Umbral de latencia de alertas
SCHOOL_HOURS	06:30-08:00,12:00-13:30	Horario escolar

Configuración por componente

Kafka

Kafka 4.3 corre en modo KRaft, con un solo nodo que es a la vez broker y controlador. Tiene dos listeners: INTERNO (kafka:9092) para los contenedores y EXTERNO (localhost:9094) solo para depurar desde el servidor. La creación automática de topics está desactivada (KAFKA_AUTO_CREATE_TOPICS_ENABLE=false), la retención es de 168 horas y, por ser un solo broker, los factores de replicación internos son 1. El contenedor kafka-init crea los topics con el script de la Figura 4.

Figura 4

Script de creación de topics

```
set -e

SERVIDOR="kafka:9092"
RETENCION_MS=604800000   # 7 días en milisegundos

crear_topic() {
  nombre=$1
  particiones=$2
  /opt/kafka/bin/kafka-topics.sh --bootstrap-server "$SERVIDOR" --create --if-not-exists \
    --topic "$nombre" --partitions "$particiones" --replication-factor 1 \
    --config retention.ms=$RETENCION_MS
  echo "Topic listo: $nombre ($particiones particiones)"
}

# 3 particiones donde hay más volumen, para poder repartir el trabajo por zona
crear_topic events.standardized 3
crear_topic events.invalid 1
crear_topic correlated.alerts 3
crear_topic external.pnc 1
crear_topic external.bomberos 1
crear_topic external.conred 1
```

Nota. Fragmento del archivo infra/kafka/crear-topics.sh, líneas 7 a 27.

NGINX

NGINX reparte las peticiones en turnos entre los tres ingestores. Si uno falla dos veces seguidas lo deja de usar 10 segundos y reintenta la petición en otro (Figura 5). Con zone y resolve, vuelve a resolver la dirección de un ingestor cuando su contenedor se reinicia, y agrega la cabecera X-Upstream con el ingestor que atendió.

Figura 5

Configuración del balanceador

```
upstream ingestores {
    # zone + resolve: NGINX vuelve a preguntar la IP de cada ingestor
    # cuando su contenedor se reinicia (Docker puede darle otra IP)
    zone ingestores 64k;
    server ingestor-1:8000 max_fails=2 fail_timeout=10s resolve;
    server ingestor-2:8000 max_fails=2 fail_timeout=10s resolve;
    server ingestor-3:8000 max_fails=2 fail_timeout=10s resolve;
    keepalive 64;   # reutiliza conexiones hacia los ingestores
}

server {
    listen 80;
    client_max_body_size 5m;   # lotes de hasta 500 eventos

    location / {
        proxy_pass http://ingestores;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_connect_timeout 2s;
        proxy_read_timeout 15s;
        # Si un ingestor no responde, se prueba el siguiente (también en POST)
        proxy_next_upstream error timeout http_502 non_idempotent;
        proxy_next_upstream_tries 3;
        # Header para demostrar el balanceo: dirección del ingestor que atendió
        add_header X-Upstream $upstream_addr always;
    }
}
```

Nota. Fragmento del archivo infra/nginx/nginx.conf, líneas 20 a 47.

PostgreSQL

Los scripts de infra/postgres/init se ejecutan solos la primera vez que se crea el volumen: 01_esquema.sql crea las nueve tablas con fechas TIMESTAMPTZ e índices por tiempo y sector, 02_roles.sql crea los usuarios app (lectura y escritura) y grafana_ro (solo lectura) leyendo sus contraseñas de las variables de entorno con \getenv, y 03_sectores.sql carga los siete sectores.

Tabla 17

Tablas de la base de datos

Tabla	Contenido	Escribe
zones	Los siete sectores	Script inicial
events	Eventos válidos (data y metadata en JSONB)	Correlador
alerts	Alertas con sus eventos de origen	Correlador
slo_violations	Alertas que superaron el umbral de latencia	Correlador
invalid_events	Eventos rechazados (dead-letter)	Despachador
dispatches	Envíos a instituciones	Despachador
daily_stats	Resumen diario de alertas	Airflow
false_positive_stats	Proporción de falsos positivos por sector	Airflow
evaluation_results	Resultados de la evaluación de escenarios	evaluar.py

Redis

Redis 7.4 se ejecuta con persistencia AOF (redis-server --appendonly yes), de modo que al reiniciarse recupera el estado del correlador: ventanas por sector, vehículos, eventos procesados, cooldowns y la lista de placas buscadas.

Grafana

Grafana 12.4 se configura por archivos (aprovisionamiento): el datasource de PostgreSQL usa el usuario grafana_ro y el de Elasticsearch apunta al índice alertas

(Figura 6); un proveedor carga los seis dashboards desde infra/grafana/dashboards. Las contraseñas se toman de variables de entorno, no del archivo. El detalle de los dashboards está en el capítulo Visualización en tiempo real.

Figura 6

Datasources aprovisionados

```
datasources:
  - name: PostgreSQL
    uid: postgres-sjs
    type: grafana-postgresql-datasource
    url: postgres:5432
    user: grafana_ro
    isDefault: true
    editable: false
    jsonData:
      database: $POSTGRES_DB
      sslmode: disable
      postgresVersion: 1600
      timescaledb: false
      maxOpenConns: 10
    secureJsonData:
      password: $GRAFANA_RO_PASSWORD

  - name: Elasticsearch
    uid: elastic-sjs
    type: elasticsearch
    url: http://elasticsearch:9200
    editable: false
    jsonData:
      index: alertas
      timeField: createdAt
```

Nota. Fragmento del archivo infra/grafana/provisioning/datasources/datasources.yml, líneas 9 a 33.

Elasticsearch y Airflow

Elasticsearch 8.19 corre como nodo único (discovery.type=single-node), con 1 GB de memoria de Java y la seguridad desactivada solo por tratarse de un laboratorio en una sola máquina, con su puerto publicado únicamente en 127.0.0.1. Airflow 3.3 corre en modo standalone: antes de arrancar, el contenedor escribe la contraseña del usuario admin desde AIRFLOW_ADMIN_PASSWORD; los DAGs se montan en solo lectura desde airflow/dags, y los ejemplos de Airflow están desactivados.

Puesta en marcha y operación

Con Docker Desktop instalado, el sistema se levanta y se opera con los comandos del manual de instalación (README) incluido en el repositorio: crear el archivo .env, levantar el perfil completo, cargar la lista de placas robadas y comprobar con los escenarios guionados que todo funciona. La primera vez se descargan unos 7 GB de imágenes.

Verificación

La configuración se verificó de dos formas: clonando el repositorio en una carpeta nueva y levantando el sistema desde cero siguiendo solo el manual (todos los servicios iniciaron y los escenarios generaron sus alertas); y revisando con docker compose ps que los servicios con chequeo de salud aparezcan como healthy.

Simulación de sensores urbanos

El proyecto no usa sensores físicos: los sensores urbanos se emulan con programas que reproducen su comportamiento, frecuencia, ubicación y tipo de evento. Este capítulo describe los scripts de simulación, cómo generan los datos de cada sensor, cómo se ejecutan y qué resultados producen. Los scripts están en la carpeta simulador del repositorio y el plan de carga de JMeter en pruebas/jmeter.

Herramientas de simulación

Tabla 18

Herramientas

Herramienta sugerida	Uso en el proyecto
Scripts Python	Herramienta principal: generan los cinco tipos de eventos, el tráfico normal, los picos de carga, los escenarios guionados y los reportes manuales
Artillery / JMeter	JMeter 5.5 para pruebas de carga HTTP de la capa de ingestión (plan pruebas/jmeter/carga.jmx)
Apache Kafka Producer CLI	No se usa para generar eventos: los eventos entran por la API REST para pasar por la validación, igual que en un despliegue real. Las herramientas de línea de comandos de Kafka se usaron para inspeccionar los topics durante las pruebas
CSV predefinidos + Randomizer	Se reemplazan por config/sectores.json (sectores con coordenadas reales, pesos y radios) y por generadores aleatorios con distribución por sector y por hora

Organización de los scripts

Tabla 19

Archivos del simulador

Archivo	Función
generadores.py	Crea los eventos de cada tipo de sensor con el contrato común, calcula el HMAC de las placas y daña eventos a propósito
envio.py	Envía lotes a NGINX con reintentos y lleva los contadores del resumen
simulador.py	Programa principal: modos normales, pico y escenario
escenarios.py	Los nueve escenarios guionados y su alerta esperada
reportar.py	Reporte ciudadano manual e interactivo
cargar_buscados.py	Carga en Redis los hashes de las placas reportadas como robadas
evaluar.py	Compara alertas esperadas y obtenidas por escenario

Todos los scripts se ejecutan dentro de un contenedor (no requieren Python instalado en la máquina) con el comando docker compose run --rm simulador seguido del script y sus opciones.

Sectores simulados

Los eventos se ubican en los siete sectores de config/sectores.json (Tabla 20). El peso es la probabilidad de que un evento ocurra en ese sector y el radio define el círculo alrededor del centro donde se dispersan los eventos, para que el mapa de calor se vea natural. Se validan los límites del municipio: latitud entre 14.60 y 14.90, longitud entre -90.75 y -90.50.

Tabla 20

Sectores del municipio

Sector	Nombre	Tipo	Latitud	Longitud	Peso	Radio	Escolar	Límite
SJS-01	Casco urbano	comercial	14.71840	-90.64340	0.20	600 m	Sí	40 km/h
SJS-02	Sajcavillá	mixto	14.70645	-90.62770	0.14	700 m	No	50 km/h
SJS-03	Cruz Blanca	residencial	14.73330	-90.66670	0.20	800 m	Sí	50 km/h
SJS-04	Lo de Mejía	corredor	14.69256	-90.60027	0.12	900 m	No	60 km/h
SJS-05	Ciudad Quetzal	urbano denso	14.70313	-90.58035	0.20	800 m	Sí	50 km/h
SJS-06	Villas del Quetzal	residencial	14.70831	-90.55144	0.09	600 m	No	40 km/h
SJS-07	Montúfar	rural	14.79956	-90.63965	0.05	1500 m	No	60 km/h

Generación de eventos por sensor

Cada evento lleva un identificador único (UUID), la hora en que ocurrió, la hora de envío (que se fija justo antes de enviarlo, para medir la latencia real), los datos del dispositivo, la ubicación y los datos propios del sensor (Tabla 21). La Figura 7 muestra un evento completo.

Tabla 21

Datos generados por tipo de sensor

Sensor	Datos y valores	Comportamiento en tráfico normal
Botón de pánico	alertType: PANICO, EMERGENCIA_MEDICA o INCENDIO; origin: BOTON_FIJO o APP_MOVIL; userRef: seudónimo solo si viene de la app	80 % pánico, 15 % emergencia médica, 5 % incendio
Cámara LPR	plateHash (HMAC-SHA256), platePartial (P-***BCD), estimatedSpeedKmh, direction (ENTRADA o SALIDA), vehículo (tipo, color, modelo)	Placa sintética guatemalteca nueva en cada lectura; 0.2 % sobre el límite
Radar de velocidad	speedKmh, limitKmh, headingDegrees, anomalyType: NINGUNA, EXCESO_VELOCIDAD o TRAYECTORIA_ANOMALA	0.2 % con exceso y 0.1 % con trayectoria anómala
Sensor acústico	soundType: DISPARO, EXPLOSION o VIDRIO_ROTO; decibels; criticalProbability	Probabilidad casi siempre por debajo de 0.7
Reporte ciudadano	reportType: ACCIDENTE, INCENDIO o ALTERCADO; message (texto libre, máximo 280 caracteres); origin: USUARIO, APP o PUNTO_FISICO; accuracyMeters	Mensajes sintéticos de ejemplo por tipo

Nota. Las anomalías son raras a propósito en el tráfico normal: con anomalías frecuentes aparecerían alertas graves por puro azar. Las alertas importantes vienen de los escenarios guionados.

Figura 7

Ejemplo de evento de botón de pánico


```
{
  "schemaVersion": "1.0",
  "eventId": "8f14e45f-ceea-4e1b-9c2a-1a2b3c4d5e6f",
  "eventType": "security.panic_button.pressed",
  "timestamp": "2026-10-28T22:05:13.412-06:00",
  "emittedAt": "2026-10-28T22:05:13.430-06:00",
  "scenarioId": null,
  "device": { "deviceId": "PAN-SJS05-012", "deviceType": "panic_button",
              "model": "SafeCity-PB100", "firmwareVersion": "1.2.0" },
  "location": { "zone": "SJS-05", "zoneName": "Ciudad Quetzal",
                "city": "San Juan Sacatepéquez",
                "latitude": 14.70391, "longitude": -90.58102 },
  "data": { "alertType": "PANICO", "origin": "APP_MOVIL", "userRef": "u_7c1e9a" },
  "metadata": { "batteryLevel": 84, "signalStrength": -67, "protocol": "MQTT" }
}
```


Nota. Contrato de eventos del sistema (device, location, data y metadata).

Tasas y comportamiento en el tiempo

Cada segundo, el simulador calcula cuántos eventos de cada tipo genera: toma la tasa por segundo, la multiplica por el multiplicador del modo y por un factor según la hora del día, y la convierte en un número entero con una moneda al aire para la parte fraccionaria (Figura 8). Hay más tráfico vehicular en las horas pico (06:00 a 09:00 y 17:00 a 20:00) y más pánicos y disparos de noche (19:00 a 02:00), con un factor de 1.5.

Figura 8

Tasas por segundo y factor horario

```
# Tasas por segundo en todo el municipio (sección 9 del plan)
TASAS = {
    "lpr": 3,
    "velocidad": 2,
    "acustico": 2 / 60,
    "panico": 1 / 60,
    "reporte": 2 / 60,
}
GENERADORES = {
    "lpr": evento_lpr, "velocidad": evento_velocidad, "acustico": evento_acustico,
    "panico": evento_panico, "reporte": evento_reporte,
}



def factor_hora(tipo, hora):
    """Más tráfico en horas pico; más pánico y disparos de noche."""
    if tipo in ("lpr", "velocidad") and (6 <= hora < 9 or 17 <= hora < 20):
        return 1.5
    if tipo in ("panico", "acustico") and (hora >= 19 or hora < 2):
        return 1.5
    return 1.0



def cuantos(tasa):
    """Cantidad de eventos en este segundo: la parte entera más una moneda al aire por la
fracción."""
    entera = int(tasa)
    return entera + (1 if random.random() < tasa - entera else 0)
```

Nota. Fragmento del archivo simulador/simulador.py, líneas 26 a 53.

Tabla 22

Tasas de simulación

Sensor	Normal (todo el municipio)	Pico (multiplicador 20)
Cámara LPR	3 por segundo	60 por segundo
Radar de velocidad	2 por segundo	40 por segundo
Sensor acústico	2 por minuto	40 por minuto
Botón de pánico	1 por minuto	20 por minuto
Reporte ciudadano	2 por minuto	40 por minuto

Eventos inválidos

Para probar la validación y la cola de mensajes muertos (dead-letter), el simulador daña a propósito el 1.5 % de los eventos con alguno de los defectos de la Tabla 23. El ingestor los rechaza con un motivo legible y los publica en el topic events.invalid.

Tabla 23

Defectos introducidos a propósito

Defecto	Motivo de rechazo esperado
Placa en claro dentro de data (privacidad)	PLACA_EN_CLARO
Sector inexistente (SJS-99)	ZONA_DESCONOCIDA
Latitud fuera del municipio	COORDENADAS_FUERA_DE_LIMITES
Fecha en el futuro (año 2099)	TIMESTAMP_EN_EL_FUTURO
Falta el bloque device	CAMPO_INVALIDO:device
Mensaje de reporte de 300 caracteres	MENSAJE_DEMASIADO_LARGO
Hash de placa con formato incorrecto	PLATE_HASH_INVALIDO
Versión de esquema desconocida	CAMPO_INVALIDO:schemaVersion

Envío y reintentos

Los eventos se envían en lotes de hasta 50 a POST /events/batch a través de NGINX, al menos una vez por segundo, con varios hilos en paralelo. Si un envío falla (por ejemplo, HTTP 503 porque Kafka no responde), se reintenta con espera creciente de 1, 2, 4 y 8 segundos, hasta cinco intentos; si aun así falla, el lote se cuenta como

perdido (Figura 9). Al final de cada ejecución se imprime un resumen con los eventos enviados, aceptados, rechazados, perdidos, los reintentos y los eventos por segundo.

Figura 9

Envío con reintentos y espera creciente

```
def enviar(eventos):
    """Envía una lista de eventos a /events/batch. Devuelve la respuesta JSON o None si se
perdió."""
    if not eventos:
        return None
    # emittedAt = momento real de envío; el correlador lo usa para medir la latencia
    momento = ahora_iso()
    for evento in eventos:
        if isinstance(evento, dict) and evento.get("emittedAt") is None:
            evento["emittedAt"] = momento

    espera = 1
    for intento in range(1, INTENTOS + 1):
        try:
            respuesta = sesion().post(f"{URL}/events/batch", json=eventos, timeout=15)
            if respuesta.status_code == 200:
                cuerpo = respuesta.json()
                sumar("enviados", len(eventos))
                sumar("aceptados", cuerpo["aceptados"])
                sumar("rechazados", cuerpo["rechazados"])
                return cuerpo
            motivo = f"HTTP {respuesta.status_code}"
        except requests.RequestException as error:
            motivo = type(error).__name__
        if intento < INTENTOS:
            log.warning("Envío fallido (%s), intento %d de %d, reintento en %d s", motivo, intento,
INTENTOS, espera)
            sumar("reintentos", 1)
            time.sleep(espera)
            espera *= 2

    log.error("Lote de %d eventos PERDIDO después de %d intentos", len(eventos), INTENTOS)
    sumar("enviados", len(eventos))
    sumar("perdidos", len(eventos))
    return None
```

Nota. Fragmento del archivo simulador/envio.py, líneas 37 a 69.

Modos de ejecución

Figura 10

Comandos de los scripts de simulación

```
# Tráfico normal (hasta Ctrl+C o durante N segundos)
docker compose run --rm simulador simulador.py normal --duracion 120

# Prueba de carga: multiplica las tasas
docker compose run --rm simulador simulador.py pico --multiplicador 20 --duracion 60 --hilos 16

# Escenarios guionados (todos a la vez o uno solo)
docker compose run --rm simulador simulador.py escenario todos --velocidad 2
docker compose run --rm simulador simulador.py escenario ESC-ROBO-01

# Utilidades
docker compose run --rm simulador cargar_buscados.py
docker compose run --rm simulador reportar.py
docker compose run --rm simulador evaluar.py --minutos 5
```

Nota. Los scripts se ejecutan en un contenedor; no se requiere Python en la máquina.

Escenarios guionados

Los escenarios reproducen incidentes concretos para demostrar y evaluar la correlación (Tabla 24). Los eventos se ubican en el centro exacto del sector y llevan un identificador de escenario (scenarioId), que el sistema solo usa para la evaluación. Con la opción --velocidad se dividen las esperas entre eventos (velocidad 2 = el doble de rápido). ESC-ROBO-01 y ESC-FALSO-01 son los dos ejemplos de la sección 2.4 del enunciado del proyecto.

Tabla 24

Escenarios guionados

Escenario	Sector	Secuencia	Alerta esperada
ESC-ROBO-01	Ciudad Quetzal	Pánico; 40 s después LPR de salida a 85 km/h; 20 s después radar a 92 km/h	ROBO_EN_CURSO
ESC-ARMADO-01	Casco urbano	Disparo con probabilidad 0.9; 30 s después pánico	ATAQUE_ARMADO
ESC-ACCIDENTE-01	Lo de Mejía	Trayectoria anómala; 60 s después reporte de accidente	ACCIDENTE_VIAL
ESC-INCENDIO-01	Villas del Quetzal	Pánico de incendio; 45 s después reporte de incendio	INCENDIO_CONFIRMADO
ESC-TRAMO-01	Lo de Mejía a Sajcavillá	Misma placa en dos cámaras con 120 s de diferencia (unos 100 km/h)	VELOCIDAD_ENTRE_CAMARAS
ESC-ESCOLAR-01	Casco urbano	Radar a 45 km/h en horario escolar	ZONA_ESCOLAR
ESC-BUSCADO-01	Cruz Blanca	LPR de una placa de la lista de robados	VEHICULO_REPORTADO
ESC-FALSO-01	Montúfar	Un solo pánico, nada más	PANICO_NO_CONFIRMADO
ESC-RUIDO-01	Sajcavillá	Pánico y vidrio roto con probabilidad 0.4	PANICO_NO_CONFIRMADO

Figura 11

Definición de dos escenarios

```
"ESC-ROBO-01": [
            (0, lambda: evento_panico("SJS-05", "PANICO", e, centro=True)),
            (40, lambda: evento_lpr("SJS-05", velocidad=85, direccion="SALIDA", escenario=e,
centro=True)),
            (20, lambda: evento_velocidad("SJS-05", 92, "EXCESO_VELOCIDAD", e, centro=True)),
        ],
        "ESC-ARMADO-01": [
            (0, lambda: evento_acustico("SJS-01", "DISPARO", 0.9, e, centro=True)),
            (30, lambda: evento_panico("SJS-01", "PANICO", e, centro=True)),
        ],
```

Nota. Fragmento del archivo simulador/escenarios.py, líneas 42 a 50.

Reporte ciudadano manual

El script reportar.py cumple la entrada manual de la sección 2.1 del enunciado del proyecto: pregunta en la terminal el sector, el tipo de evento (accidente, incendio o altercado) y un mensaje libre de hasta 280 caracteres, arma el evento con el mismo contrato y lo envía como lo haría la app del vecino. Al terminar muestra el identificador del reporte y el ingestor que lo atendió. Dos reportes de accidente en el mismo sector dentro de la ventana generan la alerta ACCIDENTE_VIAL.

Placas buscadas y evaluación

cargar_buscados.py calcula el HMAC de una lista de diez placas sintéticas reportadas como robadas y las guarda en el conjunto placas_buscadas de Redis; es el único script del simulador que habla directamente con Redis. evaluar.py compara, para cada escenario, la alerta esperada con las alertas que el correlador guardó en PostgreSQL, muestra una tabla con aciertos, faltantes y alertas adicionales, y guarda el resultado en la tabla evaluation_results.

Plan de carga con JMeter

El plan pruebas/jmeter/carga.jmx envía lotes de eventos válidos (cámaras LPR y radares) a POST /events/batch. Un preprocesador en Groovy arma cada lote con identificadores y horas reales. Los parámetros se pasan con la opción -J (Tabla 25).

Tabla 25

Parámetros del plan de JMeter

Parámetro	Valor por defecto	Significado
hilos	10	Usuarios virtuales enviando a la vez
rampa	5	Segundos para arrancar todos los hilos
duracion	60	Segundos de prueba
tamano_lote	50	Eventos por petición (máximo 500)
host / puerto	localhost / 8080	Destino (dentro de la red de Docker: nginx / 80)

Figura 12

Ejecución del plan de JMeter con Docker

```
docker run --rm --network monitoreo-sjs_default -v "${PWD}\pruebas\jmeter:/pruebas"
justb4/jmeter:5.5 `
  -n -t /pruebas/carga.jmx -Jhost=nginx -Jpuerto=80 -Jhilos=16 -Jduracion=60 -l
/pruebas/resultados.jtl
```

Nota. Comando para PowerShell documentado en el README.

Modelo de correlación e inferencia

Ubicación en la arquitectura

El correlador es un consumidor de Kafka (grupo correlador) que lee el topic events.standardized, donde los ingestores publican los eventos ya validados con la clave igual al sector. Por cada evento consulta y actualiza su estado en Redis, guarda el evento y las alertas en PostgreSQL y publica cada alerta en el topic correlated.alerts, desde donde el despachador la enruta a la PNC, Bomberos, CONRED o la aseguradora. Es decir, el correlador es consumidor y productor a la vez.

Tabla 26

Archivos del correlador

Archivo	Responsabilidad
correlador.py	Bucle principal: consume lotes de Kafka, procesa cada evento, revisa los pánicos pendientes cada 10 s y confirma el lote
reglas.py	Una función por regla (R01 a R07) y la regla de falso positivo
calculos.py	Distancia entre coordenadas (haversine), horario escolar y score de confianza
estado_redis.py	Todo lo que se guarda y consulta en Redis
base_datos.py	Escritura de eventos, alertas y violaciones de SLO en PostgreSQL

Flujo de procesamiento

La Tabla 27 relaciona cada paso con lo que hace el código.

Tabla 27

Pasos del correlador según el diagrama de referencia

Paso	Qué hace el sistema
1. Recibo evento desde Kafka	Lee lotes de hasta 500 mensajes de events.standardized con espera máxima de 0.2 s y descarta duplicados por eventId
2. Busco el vehículo o el sector en Redis	Pide los eventos recientes del sector y, si el evento es de una cámara LPR, el último paso registrado de esa placa
3. Redis responde con el último estado	Devuelve la ventana del sector (los 20 eventos más recientes de cada tipo dentro de la ventana) y el último sensor, sector y hora del vehículo
4. Correlaciono A + B	Combina el evento nuevo con los anteriores del mismo sector que no se hayan usado en una alerta del mismo tipo
5. Calculo velocidad (si aplica)	Para la regla R05, distancia entre cámaras con la fórmula de haversine dividida entre el tiempo transcurrido
6. Aplico regla de negocio	Evalúa las reglas R01 a R07; una regla solo se dispara si el evento nuevo participa en ella
7. Guardo nuevo estado en Redis	Agrega el evento a la ventana, registra el pánico como pendiente y actualiza el último paso del vehículo
8. Persisto el incidente y publico la alerta	Inserta el evento y la alerta en PostgreSQL y publica la alerta en correlated.alerts

Al terminar cada lote, el orden de confirmación es fijo: primero se confirma la transacción de PostgreSQL, luego se espera la confirmación de Kafka para las alertas publicadas, después se marcan los eventos como procesados en Redis y al final se confirma el offset en Kafka. Este orden garantiza la entrega al menos una vez: si el proceso muere en cualquier punto, el lote se vuelve a leer y lo que ya se había hecho se reconoce y se omite.

Entrada: el evento estandarizado

Cada evento sigue un sobre común con el campo schemaVersion, un identificador único (eventId), el tipo de evento, la hora en que ocurrió (timestamp) y la hora en que se envió (emittedAt), además de los bloques device, location, data y metadata. El ingestor agrega el bloque ingestion con la hora de recepción y el ingestor que lo atendió. El contenido de data depende del sensor (Tabla 21).

El campo scenarioId solo lo traen los escenarios de prueba. El correlador nunca lo usa para decidir; únicamente lo copia a la alerta cuando todos los eventos de origen pertenecen al mismo escenario, para que la herramienta de evaluación pueda medir aciertos y falsos positivos.

Estado en Redis

Redis es la memoria de corto plazo del correlador. Todo el estado necesario para correlacionar vive ahí y no dentro del proceso, lo que permite reiniciar el correlador sin perder el contexto y correr varias copias en paralelo. Redis guarda su contenido en disco con el modo AOF (append only file).

Tabla 28

Claves de Redis usadas por el correlador

Clave	Tipo	Contenido y vigencia
evento: {eventId}	texto	Marca de evento ya procesado (descarte de duplicados); 1 hora
ventana: {sector}:{tipo}	sorted set	Eventos recientes relevantes del sector, ordenados por la hora del evento; se recorta a la ventana y expira a las dos ventanas
vehiculo: {plateHash}	texto (JSON)	Último paso del vehículo: cámara, sector, hora y coordenadas; 1 hora
panicos_pendientes	sorted set	Pánicos que todavía no tienen confirmación, por hora del evento
pánico: {eventId}	texto (JSON)	Resumen del pánico pendiente, para generar el falso positivo
usado: {tipo Alerta}:{eventId}	texto	Evento ya usado en una alerta de ese tipo; dos ventanas
cooldown: {regla}:{sujeto}	texto	Bloquea repetir la misma alerta para el mismo sector o placa durante una ventana
placas_buscadas	set	Hashes de las placas reportadas como robadas (los carga cargar_buscados.py)

Ventana de tiempo

La ventana de correlación (`CORRELATION_WINDOW_SECONDS`) es de 300 s en operación normal y de 60 s en la demostración. Se mide con la hora en que ocurrió el evento y no con el reloj del servidor: así, cuando el correlador procesa un atraso después de una caída, los eventos de ese atraso se siguen correlacionando entre sí como si hubieran llegado a tiempo.

Solo entran a la ventana los eventos que alguna regla busca en ella: pánicos, sensores acústicos, reportes, lecturas de velocidad con anomalía y lecturas LPR de salida. Las lecturas normales de velocidad y las entradas LPR no se guardan, porque

las reglas que las usan (R05, R06 y R07) trabajan solo con el evento nuevo, el vehículo o la lista de placas buscadas. Además, de cada tipo se leen como máximo los 20 eventos más recientes. Esta decisión subió la capacidad del correlador de 66 a 556 eventos por segundo, porque antes cada evento leía y decodificaba miles de eventos de la ventana bajo carga.

Reglas de correlación

Las reglas son determinísticas: dadas las mismas entradas, producen siempre la misma salida, y cada alerta indica qué regla la generó y cuáles fueron sus eventos de origen (`sourceEventIds`), lo que permite auditarla hasta el sensor. Salvo R05, todas exigen el mismo sector y que los eventos estén dentro de la ventana.

Tabla 29

Reglas de correlación del sistema

Regla	Alerta	Condición	Severidad
R01	ROBO_EN_CURSO	Pánico (PANICO) + lectura LPR con dirección SALIDA + anomalía de velocidad (radar con EXCESO_VELOCIDAD o la misma cámara sobre el límite del sector)	CRITICA
R02	ATAQUE_ARMADO	Sonido DISPARO con probabilidad mayor o igual a 0.7 + pánico	CRITICA
R03	ACCIDENTE_VIAL	Reporte ACCIDENTE + radar con TRAYECTORIA_ANOMALA o EXCESO_VELOCIDAD; o bien dos reportes ACCIDENTE	ALTA
R04	INCENDIO_CONFIRMADO	Pánico de tipo INCENDIO + reporte INCENDIO o sonido EXPLOSION	CRITICA
R05	VELOCIDAD_ENTRE_CAMARAS	Misma placa (mismo hash) en dos cámaras distintas en menos de una hora; velocidad promedio del tramo mayor a 80 km/h	MEDIA (ALTA si alguna cámara está en zona escolar en horario escolar)
R06	ZONA_ESCOLAR	Lectura de radar mayor a 30 km/h en un sector escolar dentro del horario escolar	MEDIA
R07	VEHICULO_REPORTADO	El hash de la placa está en el conjunto placas_buscadas	ALTA
FP	PANICO_NO_CONFIRMADO	Pánico que no participó en ninguna alerta al cerrarse su ventana	BAJA (estado FALSO_POSITIVO_PROBABLE)

Selección de eventos

Cuando varias combinaciones cumplen una regla, se usa el evento más reciente de cada tipo que todavía no se haya usado en una alerta del mismo tipo. En R01 se prefiere una lectura LPR de salida que ya venga sobre el límite; si no la hay, se exige además un radar con exceso de velocidad. Una regla solo se dispara si el evento

recién llegado participa en ella, para que una combinación vieja no vuelva a generar la alerta con cada evento que llega.

Cálculo de velocidad entre cámaras

La regla R05 calcula la distancia entre las coordenadas de las dos cámaras con la fórmula de haversine (Sinnott, 1984), que da la distancia sobre la superficie de la Tierra a partir de latitud y longitud, y la divide entre el tiempo transcurrido entre las dos lecturas. En el escenario de prueba, una placa pasa por una cámara en Lo de Mejía y 120 s después por otra en Sajcavillá: 3.33 km en 120 s equivalen a 100 km/h, por encima del umbral de 80 km/h. La lectura de un vehículo solo se guarda si es más reciente que la registrada, porque al procesar un atraso de particiones distintas pueden llegar antes lecturas nuevas.

Horario escolar

El horario escolar se configura con SCHOOL_HOURS (por defecto de 06:30 a 08:00 y de 12:00 a 13:30) y se evalúa con la hora del evento en Guatemala (UTC-6, sin horario de verano). Los sectores escolares son Casco urbano, Cruz Blanca y Ciudad Quetzal.

Inferencia: score de confianza

Cada alerta incluye un score entre 0 y 1 que resume qué tan confiable es la inferencia. Se calcula como la proporción de condiciones cumplidas multiplicada por el promedio de la confianza de los sensores que participaron:

score = (condiciones cumplidas / condiciones de la regla) × promedio de la confianza de los eventos de origen

Tabla 30

Confianza asignada a cada tipo de sensor

Sensor	Confianza	Razón
Acústico	Su criticalProbability	El propio sensor estima la probabilidad de un evento crítico
Botón de pánico fijo	0.90	Dispositivo instalado y controlado por la municipalidad
Pánico desde la app	0.80	Más expuesto a toques accidentales
Reporte ciudadano	0.80 si la precisión es de 50 m o menos; 0.70 en otro caso	La ubicación aproximada reduce la certeza
Cámara LPR y radar	0.95	Medición automática de alta precisión

Como las alertas se generan cuando todas las condiciones se cumplen, el primer factor vale 1, salvo en el falso positivo, donde vale 1/2 (hubo pánico, pero nada lo confirmó). Por ejemplo, la alerta de ataque armado del escenario de prueba combina un disparo con probabilidad 0.9 y un pánico: si el pánico viene de un botón fijo (0.90), el score es 0.90; si viene de la app (0.80), es 0.85. Un falso positivo de un pánico de botón fijo tiene score 0.45.

Control de repeticiones y duplicados

- **Duplicados de entrada:** si llega dos veces el mismo eventId (por un reintento del cliente o de NGINX), se descarta; además, la tabla de eventos usa ON CONFLICT DO NOTHING.
- **Eventos usados:** un evento que participó en una alerta de un tipo no se reutiliza para otra alerta del mismo tipo.
- **Cooldown:** cuando una regla se dispara, no vuelve a dispararse para el mismo sujeto durante una ventana. El sujeto es el sector en R01 a R04 y R06, y la placa en R05 y R07, porque dos vehículos distintos en el mismo sector son dos alertas distintas.
- **Atomicidad:** el cooldown se toma con SET NX y el falso positivo se retira de la lista con ZREM, operaciones atómicas de Redis que impiden que dos copias del correlador generen la misma alerta.

Detección y análisis de falsos positivos

Cada pánico entra a la lista panicos_pendientes. Si participa en una alerta (robo, ataque armado o incendio), sale de la lista porque quedó confirmado. Cada 10 s, el correlador revisa los pánicos cuya ventana ya se cerró y genera para cada uno una alerta PANICO_NO_CONFIRMADO con estado FALSO_POSITIVO_PROBABLE y severidad baja. Estas alertas no se envían a ninguna institución: solo se registran.

El escenario ESC-FALSO-01 (un pánico aislado en Montúfar) reproduce el segundo ejemplo del enunciado del proyecto, y ESC-RUIDO-01 demuestra que un vidrio roto con probabilidad baja (0.4) no basta para una alerta crítica. Además, el proceso periódico analisis_falsos_positivos de Airflow calcula cada hora, por sector, la proporción de pánicos no confirmados frente a los confirmados en las últimas 24 horas, y el dashboard 3 de Grafana la muestra. Esto permite detectar sectores con botones defectuosos o mal uso de la app.

Salida: la alerta

Cada alerta se guarda en la tabla alerts y se publica en correlated.alerts con la clave igual al sector. Sus campos son: identificador, tipo, severidad, estado, sector y nombre del sector, coordenadas, score, regla, descripción en lenguaje natural, eventos de origen, escenario (si aplica), hora de creación y latencia en milisegundos.

Objetivo de nivel de servicio (SLO) de latencia

La latencia de una alerta es la hora de creación menos el emittedAt del último evento que completó la regla. El objetivo es menos de 2 000 ms (SLO_LATENCIA_MS). Si se supera, el correlador escribe una advertencia en el log y registra la alerta en la tabla slo_violations, que el dashboard 2 de Grafana muestra en rojo. Los falsos positivos no cuentan, porque por diseño esperan la ventana completa; su latencia se mide desde el cierre de la ventana.

Visualización en tiempo real

Este capítulo describe la visualización en tiempo real del sistema de monitoreo: cómo está configurado Grafana, qué muestra cada dashboard, de dónde sale cada dato y cómo se usa durante la operación y la demostración.

Configuración

Tabla 31

Configuración de Grafana

Elemento	Valor
Versión	Grafana 12.4.12 (imagen grafana/grafana:12.4.12)
Acceso	http://localhost:3000 o http://<IP del servidor>:3000, usuario admin (contraseña en el archivo .env)
Carpeta	Monitoreo SJS, con seis dashboards
Datasource principal	PostgreSQL 16 con el usuario grafana_ro, que solo puede leer
Datasource de búsqueda	Elasticsearch 8.19, índice alertas (perfil completo)
Aprovisionamiento	Datasources y dashboards definidos en archivos versionados (infra/grafana); se cargan solos al iniciar
Actualización	Refresco automático cada 5 segundos (30 s en el histórico)
Zona horaria	América/Guatemala
Mapa base	OpenStreetMap; lo descarga el navegador de quien mira el dashboard

Que los dashboards estén definidos como archivos tiene dos ventajas: se recrean idénticos en cualquier instalación y quedan versionados junto con el código. Después de editar uno de esos archivos se debe reiniciar Grafana (docker compose restart grafana) para que tome el cambio.

Casos de uso cubiertos

Tabla 32

Casos de uso de visualización del enunciado del proyecto

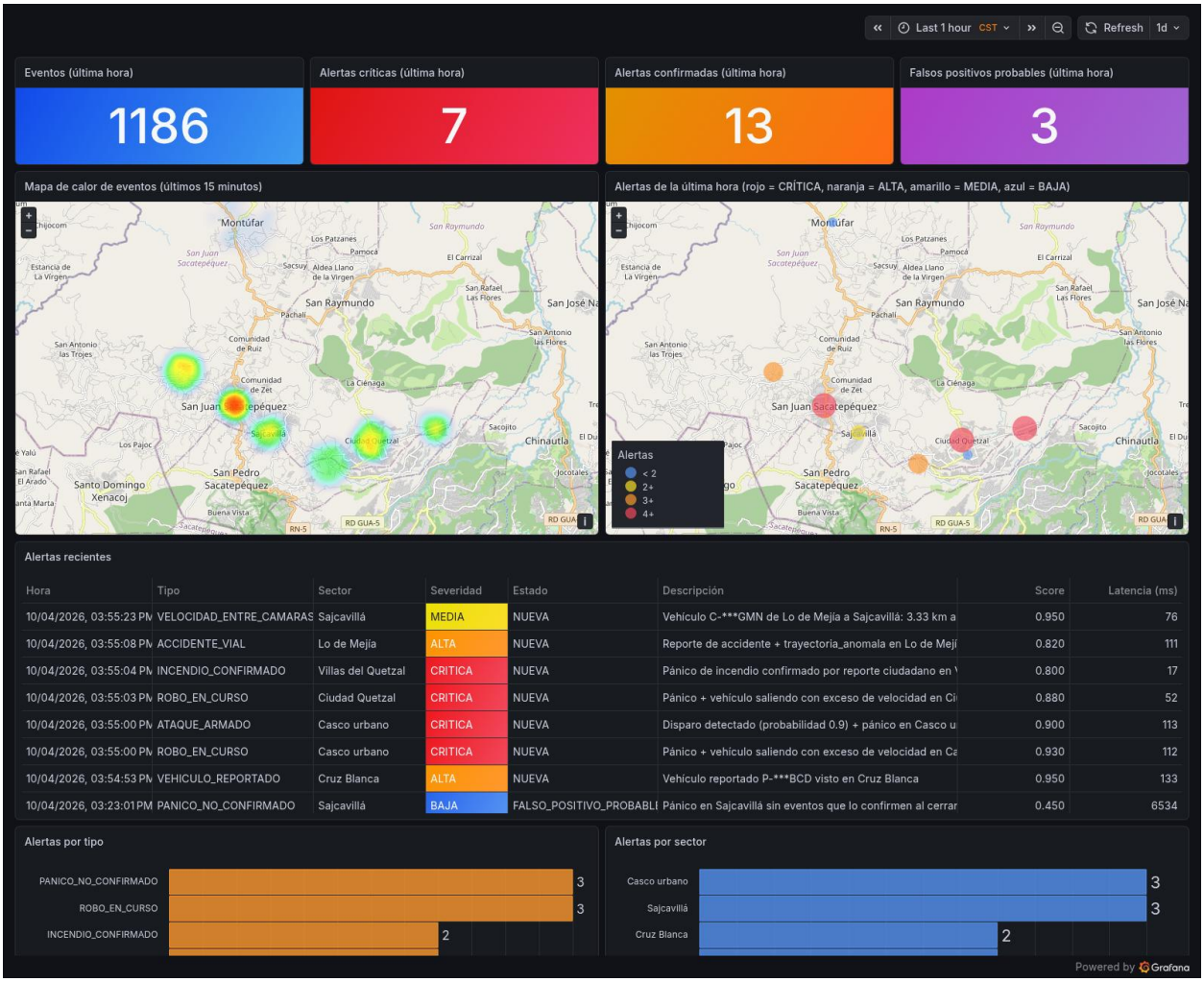
Caso de uso (sección 7)	Dónde se cumple
Mapas de calor	Dashboard 1: mapa de calor de eventos de los últimos 15 minutos sobre San Juan Sacatepéquez
Listas de incidentes	Dashboard 1: tabla de alertas recientes; Dashboard 4: bandeja de cada institución; Dashboard 5: alertas enviadas; Dashboard 3: búsqueda histórica
Alertas críticas	Dashboard 1: contador de alertas críticas, severidad en color en la tabla y en el mapa
Falsos positivos y validación	Dashboard 1: contador de falsos positivos; Dashboard 3: proporción de falsos positivos por sector; Dashboard 5: pánicos confirmados y falsos positivos reales con su motivo; Dashboard 6: recorrido de un pánico
Procesamiento en tiempo real y SLOs	Dashboard 2: eventos por segundo, latencia p50 y p95, inválidos y violaciones de SLO

Dashboard 1: Centro de Monitoreo San Juan Sacatepéquez

Es la pantalla del operador municipal (Figura 13). Arriba muestra cuatro contadores de la última hora: eventos recibidos, alertas críticas, alertas confirmadas y falsos positivos probables. Debajo, dos mapas sobre el municipio: el de la izquierda es el mapa de calor de los eventos de los últimos 15 minutos, que muestra dónde se concentra la actividad; el de la derecha marca cada alerta de la última hora con un punto cuyo color indica la severidad (rojo crítica, naranja alta, amarillo media y azul baja). Después viene la tabla de alertas recientes, con la severidad en color, la descripción, el score y la latencia, y al final las barras de alertas por tipo y por sector.

Figura 13

Dashboard 1: Centro de Monitoreo San Juan Sacatepéquez



Nota. Captura del sistema en funcionamiento con tráfico normal.

Tabla 33

Paneles del dashboard 1

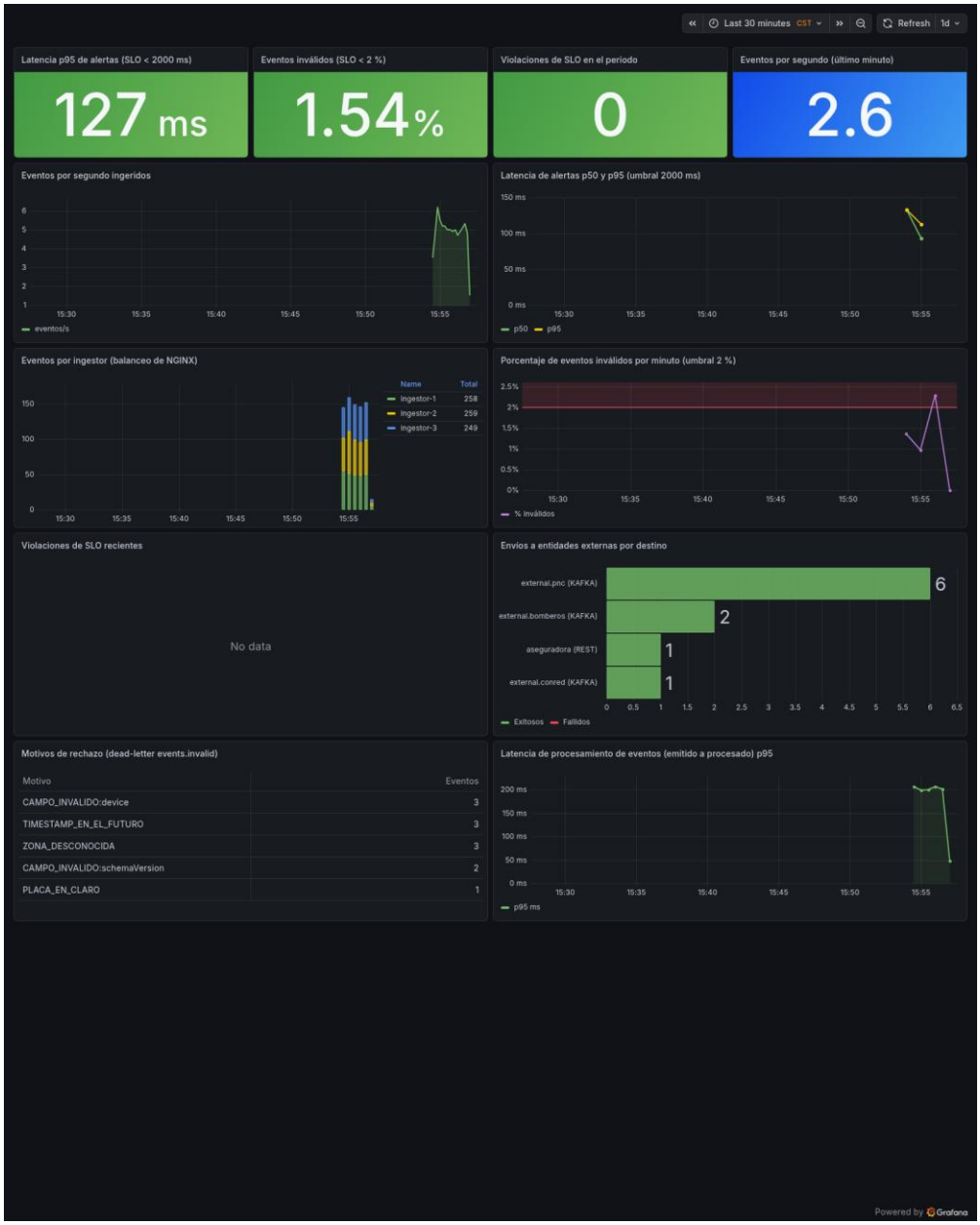
Panel	Visualización	Origen de los datos
Eventos (última hora)	Indicador	PostgreSQL: events
Alertas críticas (última hora)	Indicador	PostgreSQL: alerts
Alertas confirmadas (última hora)	Indicador	PostgreSQL: alerts
Falsos positivos probables (última hora)	Indicador	PostgreSQL: alerts
Mapa de calor de eventos (últimos 15 minutos)	Mapa	PostgreSQL: events
Alertas de la última hora (rojo = CRÍTICA, naranja = ALTA, amarillo = MEDIA, azul = BAJA)	Mapa	PostgreSQL: alerts, zones
Alertas recientes	Tabla	PostgreSQL: alerts, zones
Alertas por tipo	Barras	PostgreSQL: alerts
Alertas por sector	Barras	PostgreSQL: zones, alerts

Dashboard 2: Operación y SLOs

Es la pantalla técnica (Figura 14). Muestra si el sistema cumple sus objetivos de servicio: la latencia p95 de las alertas con el umbral de 2 000 ms (el indicador se pone en rojo si se supera), el porcentaje de eventos inválidos con el umbral de 2 %, las violaciones de SLO del periodo y los eventos por segundo. Las series de tiempo muestran la ingesta, la latencia p50 y p95 con la línea roja del umbral, el reparto de eventos entre los tres ingestores (que demuestra el balanceo de NGINX) y el porcentaje de inválidos por minuto. También lista las violaciones de SLO recientes, los envíos a cada institución, los motivos de rechazo de la cola de mensajes muertos y la latencia de procesamiento de los eventos.

Figura 14

Dashboard 2: Operación y SLOs



Nota. Captura del sistema en funcionamiento.

Tabla 34

Paneles del dashboard 2

Panel	Visualización	Origen de los datos
Latencia p95 de alertas (SLO < 2000 ms)	Indicador	PostgreSQL: alerts
Eventos inválidos (SLO < 2 %)	Indicador	PostgreSQL: invalid_events, events
Violaciones de SLO en el periodo	Indicador	PostgreSQL: slo_violations
Eventos por segundo (último minuto)	Indicador	PostgreSQL: events
Eventos por segundo ingeridos	Serie de tiempo	PostgreSQL: events
Latencia de alertas p50 y p95 (umbral 2000 ms)	Serie de tiempo	PostgreSQL: alerts
Eventos por ingestor (balanceo de NGINX)	Serie de tiempo	PostgreSQL: events
Porcentaje de eventos inválidos por minuto (umbral 2 %)	Serie de tiempo	PostgreSQL: events, invalid_events
Violaciones de SLO recientes	Tabla	PostgreSQL: slo_violations, alerts, zones
Envíos a entidades externas por destino	Barras	PostgreSQL: dispatches
Motivos de rechazo (dead-letter events.invalid)	Tabla	PostgreSQL: invalid_events
Latencia de procesamiento de eventos (emitido a procesado) p95	Serie de tiempo	PostgreSQL: events

Tabla 35

Umbrales configurados en el dashboard 2

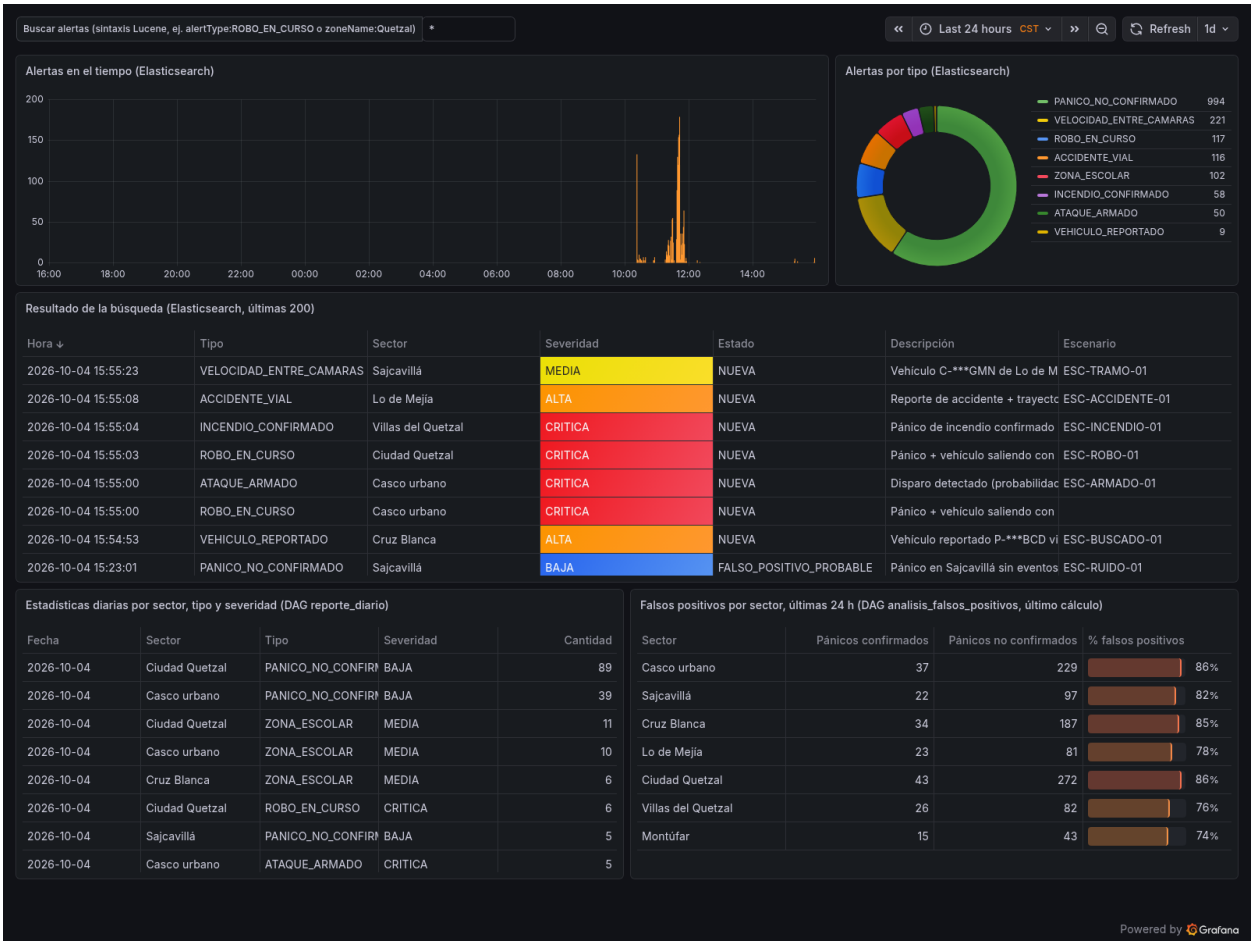
Indicador	Umbral	Qué indica el color rojo
Latencia p95 de alertas	2 000 ms	Las alertas tardan más de lo prometido (SLO de latencia)
Eventos inválidos	2 %	Demasiados eventos rechazados: posibles sensores dañados o mal configurados
Violaciones de SLO	1 o más	Hubo al menos una alerta que superó el umbral en el periodo

Dashboard 3: Histórico

Disponible en el perfil completo (Figura 15). Permite buscar alertas en Elasticsearch con un cuadro de texto que acepta la sintaxis de búsqueda Lucene (por ejemplo alertType:ROBO_EN_CURSO o zoneName:Quetzal) y muestra la cantidad de alertas en el tiempo, su distribución por tipo y el resultado de la búsqueda. Además, presenta las estadísticas calculadas por Airflow: el resumen diario por sector, tipo y severidad, y la proporción de falsos positivos por sector en las últimas 24 horas, que es el análisis de falsos positivos que el enunciado del proyecto considera mérito adicional.

Figura 15

Dashboard 3: Histórico



Nota. Captura del sistema en funcionamiento (perfil completo).

Tabla 36

Paneles del dashboard 3

Panel	Visualización	Origen de los datos
Alertas en el tiempo (Elasticsearch)	Serie de tiempo	Elasticsearch (índice alertas)
Alertas por tipo (Elasticsearch)	Anillo	Elasticsearch (índice alertas)
Resultado de la búsqueda (Elasticsearch, últimas 200)	Tabla	Elasticsearch (índice alertas)
Estadísticas diarias por sector, tipo y severidad (DAG reporte_diario)	Tabla	PostgreSQL: daily_stats, zones
Falsos positivos por sector, últimas 24 h (DAG analisis_falsos_positivos, último cálculo)	Tabla	PostgreSQL: false_positive_stats, zones

Dashboard 4: Bandeja de instituciones

Muestra una tabla por institución externa (PNC, Bomberos, CONRED y aseguradora) con las alertas que recibió cada una, las más recientes primero, a partir de la tabla dispatches (Figura 16). Sirve para comprobar que el despachador enruta correctamente: por ejemplo, CONRED solo debe recibir alertas de incendio confirmado, y la aseguradora solo accidentes viales con su número de caso. Al hacer clic en una celda se ve el texto completo de la descripción.

Figura 16

Dashboard 4: Bandeja de instituciones



Nota. Captura del sistema en funcionamiento.

Tabla 37

Paneles del dashboard 4

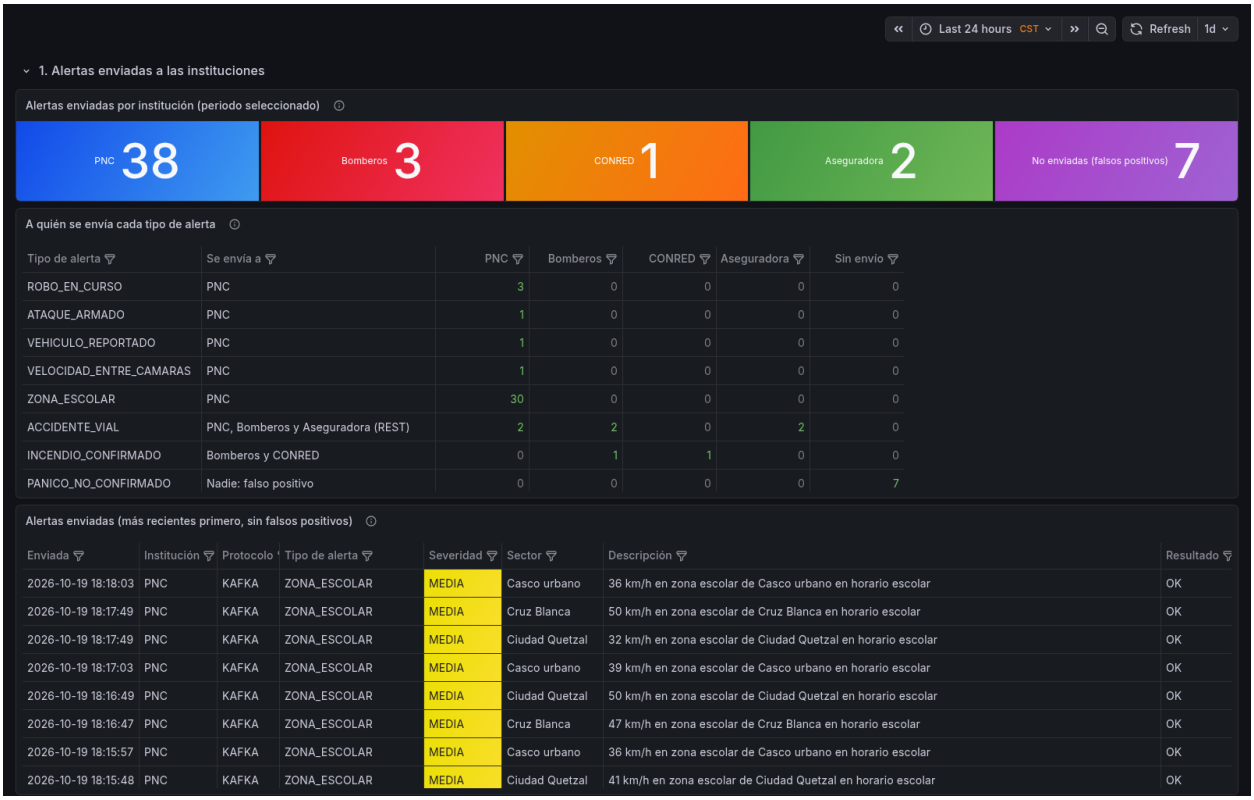
Panel	Visualización	Origen de los datos
PNC (topic external.pnc), últimas 200	Tabla	PostgreSQL: dispatches, alerts, zones
Bomberos (topic external.bomberos), últimas 200	Tabla	PostgreSQL: dispatches, alerts, zones
CONRED (topic external.conred), últimas 200	Tabla	PostgreSQL: dispatches, alerts, zones
Aseguradora (REST /siniestros), últimas 200	Tabla	PostgreSQL: dispatches, alerts, zones

Dashboard 5: Despacho y falsos positivos

Muestra a quién avisa el sistema y qué pasa con cada pánico (Figuras 17 y 18). La primera parte cuenta las alertas enviadas a cada institución en el periodo, compara la regla del despachador (a quién se envía cada tipo de alerta) con los envíos reales y lista los envíos, que se pueden filtrar por institución desde el encabezado de la columna. Los falsos positivos no aparecen en esa lista porque no se envían a ninguna institución. La segunda parte explica cómo un pánico pasa a alerta real y separa el resultado de los pánicos del periodo: los confirmados por una regla (R01, R02 o R04), los falsos positivos reales y los falsos positivos corregidos, que son pánicos marcados como falsos cuando el correlador iba atrasado y confirmados después. De cada pánico confirmado indica qué evento lo confirmó, qué regla se cumplió, la demora y a quién se avisó; de cada falso positivo real, qué otros eventos hubo en el sector y por qué no sirvieron (Tabla 38).

Figura 17

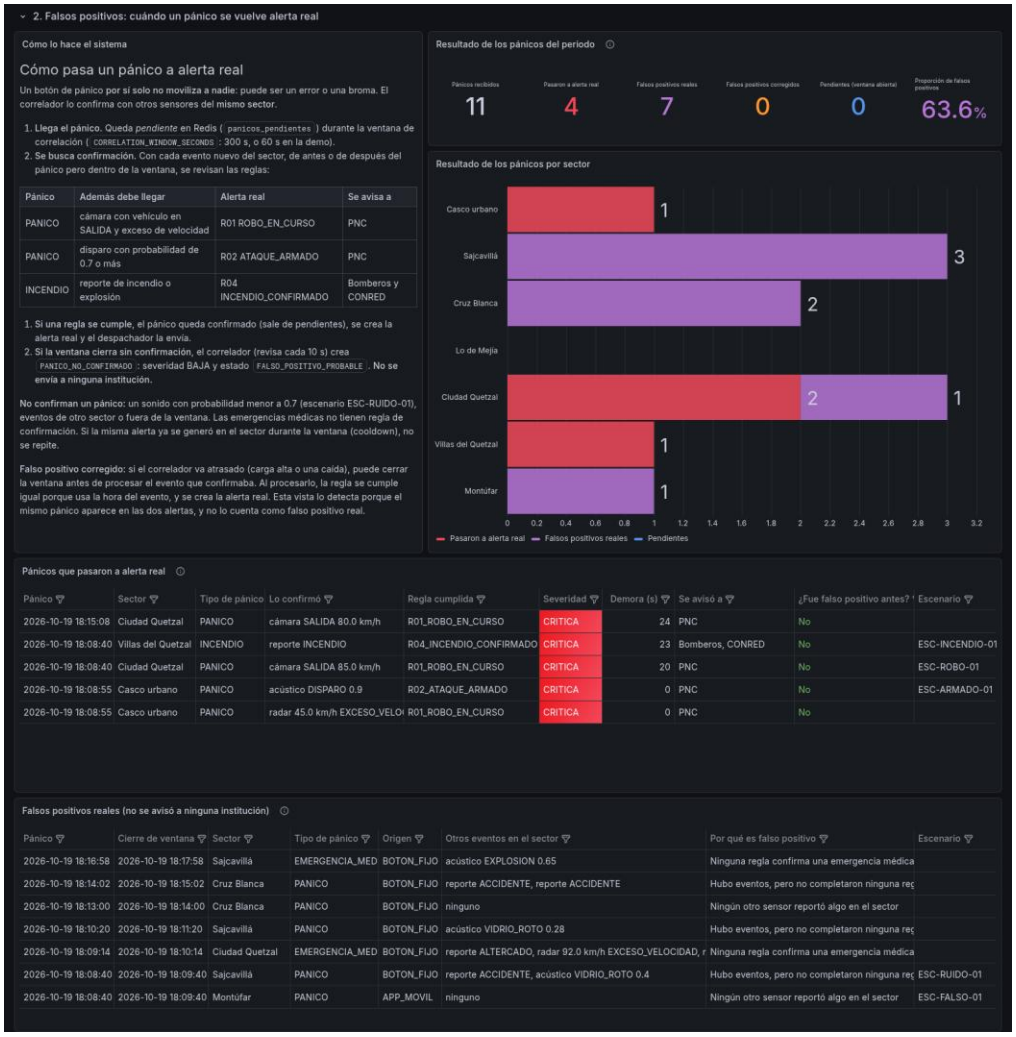
Dashboard 5: alertas enviadas a las instituciones



Nota. Captura del sistema en funcionamiento.

Figura 18

Dashboard 5: pánicos confirmados y falsos positivos



Nota. Captura del sistema en funcionamiento.

Tabla 38

Paneles del dashboard 5

Panel	Visualización	Origen de los datos
Alertas enviadas por institución (periodo seleccionado)	Indicador	PostgreSQL: dispatches
A quién se envía cada tipo de alerta	Tabla	PostgreSQL: alerts, dispatches
Alertas enviadas (más recientes primero, sin falsos positivos)	Tabla	PostgreSQL: dispatches, alerts, zones
Cómo lo hace el sistema	Texto	Explicación fija
Resultado de los pánicos del periodo	Indicador	PostgreSQL: events, alerts
Resultado de los pánicos por sector	Barras	PostgreSQL: events, alerts, zones
Pánicos que pasaron a alerta real	Tabla	PostgreSQL: events, alerts, dispatches, zones
Falsos positivos reales (no se avisó a ninguna institución)	Tabla	PostgreSQL: events, alerts, zones

Dashboard 6: Recorrido de un pánico

Sirve para explicar el sistema con un caso concreto. Arriba se elige un pánico de la lista, el más reciente primero, y con el selector Mostrar se filtran solo las alertas reales, solo los falsos positivos o los pendientes. El dashboard muestra el camino que siguió ese pánico: un diagrama con una caja por servicio (sensor, NGINX e ingestor, Kafka, correlador y Redis, decisión, despachador e institución), coloreada según por dónde pasó (verde si pasó por ahí, rojo si terminó en alerta real y naranja si fue falso positivo), con la hora y los milisegundos de cada tramo; y una línea de tiempo paso a paso que explica qué hizo cada servicio, qué otro sensor confirmó el pánico, aunque haya llegado antes, o, si fue falso positivo, qué eventos hubo en el sector y por qué no

sirvieron. Al final incluye dos textos de apoyo: qué hace cada pieza del sistema y cuándo un pánico pasa a alerta real (Tabla 39).

Tabla 39

Paneles del dashboard 6

Panel	**Visualización**	**Origen de los datos**
Mostrar y Pánico	Listas de selección	PostgreSQL: events, alerts
Resumen del pánico elegido	Indicador	PostgreSQL: events, alerts, dispatches, zones
El camino del pánico por el sistema	Diagrama (canvas)	PostgreSQL: events, alerts, dispatches
Línea de tiempo, paso a paso	Tabla	PostgreSQL: events, alerts, dispatches, zones
Cómo leer este tablero, Qué hace cada pieza y Cuándo un pánico pasa a alerta real	Texto	Explicación fija

Evaluación de resultados

La evaluación se hizo en el servidor del proyecto (Intel i7-13650HX, 32 GB de RAM, 20 GB y 12 núcleos asignados a Docker) con el perfil completo del sistema. Se midieron la exactitud de la correlación, la latencia, la capacidad máxima y el comportamiento ante fallas.

Correlación

El simulador incluye nueve escenarios con su alerta esperada. El script evaluar.py compara las alertas esperadas con las obtenidas (por scenarioId), guarda el resultado en la tabla evaluation_results y muestra los aciertos, los faltantes y las alertas adicionales. La Tabla 40 muestra la evaluación de la versión final, con ventana de 60 s y los escenarios acelerados cuatro veces.

Tabla 40

Resultado de la evaluación de escenarios

Escenario	Secuencia simulada	Alerta esperada	Resultado	Latencia
ESC-ROBO-01	Pánico; LPR de salida a 85 km/h; radar a 92 km/h en Ciudad Quetzal	ROBO_EN_CURSO	Acierto	140 ms
ESC-ARMADO-01	Disparo con probabilidad 0.9; pánico en Casco urbano	ATAQUE_ARMADO	Acierto	30 ms
ESC-ACCIDENTE-01	Trayectoria anómala; reporte de accidente en Lo de Mejía	ACCIDENTE_VIAL	Acierto	158 ms
ESC-INCENDIO-01	Pánico de incendio; reporte de incendio en Villas del Quetzal	INCENDIO_CONFIRMADO	Acierto	98 ms
ESC-TRAMO-01	Misma placa en Lo de Mejía y Sajcavillá a unos 100 km/h	VELOCIDAD_ENTRE_CAMARAS	Acierto	185 ms
ESC-ESCOLAR-01	Radar a 45 km/h en Casco urbano en horario escolar	ZONA_ESCOLAR	Acierto	115 ms
ESC-BUSCADO-01	LPR de una placa reportada como robada en Cruz Blanca	VEHICULO_REPORTADO	Acierto	110 ms
ESC-FALSO-01	Un pánico aislado en Montúfar	PANICO_NO_CONFIRMADO	Acierto	8.9 s*
ESC-RUIDO-01	Pánico y vidrio roto con probabilidad 0.4 en Sajcavillá	PANICO_NO_CONFIRMADO	Acierto	8.9 s*

Nota. Resultado: 9 de 9 aciertos. *La latencia de los falsos positivos se mide desde el cierre de la ventana y depende de la revisión cada 10 s; no cuenta para el SLO. ESC-ROBO-01 genera además una alerta ZONA_ESCOLAR correcta, porque el radar de 92 km/h está en un sector escolar.

Durante la validación se identificaron tres comportamientos que conviene conocer al operar el sistema. Primero, con una ventana de 60 s el escenario de accidente queda en el límite, porque sus eventos están separados exactamente 60 s; en la demostración se ejecuta acelerado. Segundo, inmediatamente después de una prueba de carga muy alta, los cooldowns activados por correlaciones al azar bloquean durante una ventana las alertas de los escenarios en esos sectores. Tercero, la regla R01 es sensible al ruido: en tráfico normal casi siempre hay algún vehículo saliendo, por lo que en la práctica basta un pánico y un exceso de velocidad en el mismo sector; por eso el simulador mantiene raras las anomalías del tráfico normal.

Además, se verificó que el sistema es reproducible: al clonar el repositorio en una carpeta nueva y seguir solo el manual, el sistema arrancó desde cero y los escenarios volvieron a generar sus alertas.

Latencia y capacidad

Tabla 41

Resultados de las pruebas de carga

Prueba	Eventos/s	Perdidos	Latencia de alertas p95	Cumple el SLO
Tráfico normal	5	0	menos de 200 ms	Sí
Pico ×100 durante 2 minutos	508	0	976 ms	Sí
Pico ×130 durante 90 s	661	0	13 668 ms	No
Pico ×160 durante 90 s	813	0	42 847 ms	No
Ingestión con JMeter (sin correlador)	26 500 o más	0	No aplica	No aplica

Nota. Cuando la carga supera la capacidad del correlador no se pierden eventos: esperan en Kafka y se procesan después, por lo que solo se incumple la latencia. El límite de la ingestión lo puso el generador de carga, no el sistema.

El máximo sostenido de extremo a extremo cumpliendo el objetivo de 2 segundos es de unos 510 eventos por segundo, alrededor de 100 veces el tráfico normal estimado para el municipio. El cuello de botella es el correlador, un proceso

Python que consulta Redis varias veces por evento: con la carga máxima, el servidor completo usó menos de un núcleo de doce y unos 4.6 GB de 20 GB de memoria. Durante las pruebas se detectó que la primera versión del correlador procesaba solo 66 eventos por segundo, porque leía toda la ventana del sector con cada evento; la optimización llevó a 660, diez veces más. El diagrama de referencia menciona más de 10 000 solicitudes por segundo: la capa de ingestión supera esa cifra, mientras que la correlación de extremo a extremo se queda en unos 510 eventos por segundo, suficiente para la escala del cliente.

Tabla 42

Capacidad del correlador

Versión o configuración	Eventos por segundo
Primera versión (leía toda la ventana del sector con cada evento)	66
Ventana por tipo de evento, solo eventos relevantes, máximo 20 por tipo	556
Lo anterior más inserción en PostgreSQL por lote	660
Tres correladores en paralelo (uno por partición)	743
Máximo sostenido de extremo a extremo cumpliendo el SLO de 2 s	unos 510

Nota. Con tres correladores la mejora es limitada porque la partición más cargada concentró el 60 % de los eventos: la clave de partición es el sector y los sectores tienen tráfico distinto.

Resiliencia

Tabla 43

Resultados de las pruebas de resiliencia

Falla provocada	Eventos perdidos	Alertas duplicadas	Observación
Caída de un ingestor (20 s)	0	0	NGINX reintentó en otro ingestor; el cliente vio un solo reintento
Reinicio del correlador	0	0	Apagado ordenado; de vuelta en menos de 1 s
Correlador detenido con 15 000 eventos en cola	0	0	Kafka los conservó y se procesaron al volver
Reinicio de Redis	0	0	Reconexión automática; latencia máxima de 3.2 s
Kafka caído 32 s	0	0	Los ingestores respondieron 503 y el cliente reintentó
Kafka caído 70 s	unos 300 de 13 215 (2.3 %)	0	Algunos lotes agotaron sus cinco reintentos

Kafka es el único componente cuya caída prolongada puede perder eventos. Para eliminar esa pérdida, los sensores reales deberían guardar los eventos y reenviarlos al recuperarse la conexión.

Calidad de datos y SLOs

El porcentaje de eventos inválidos se mantuvo entre 1.4 % y 1.5 %, por debajo del umbral de 2 %, y todos correspondieron a los daños introducidos a propósito por el simulador, lo que confirma que la validación no rechaza eventos correctos. Cada violación del SLO de latencia quedó registrada en la tabla slo_violations y visible en el dashboard 2.

Comparación con casos reales de ciudades inteligentes

Centro de Operaciones de Río de Janeiro

En 2010, Río de Janeiro inauguró un centro de operaciones desarrollado con IBM que integra en una sola sala la información de decenas de agencias municipales, entre ellas video de tránsito, lluvias, accidentes y emergencias, para coordinar la respuesta de la ciudad (Singer, 2012; C40 Cities, s.f.). Nuestro sistema sigue la misma idea de integrar fuentes heterogéneas en un centro de monitoreo con visualización unificada y avisos a varias instituciones, pero a una escala y un costo adecuados para un municipio mediano: un solo servidor y software de código abierto en lugar de una instalación dedicada y contratos con un proveedor.

Barcelona

La experiencia de Barcelona muestra que una ciudad inteligente no depende solo de sensores y plataformas, sino de la gobernanza: de la coordinación entre áreas municipales, de una estrategia clara y de la capacidad de la institución para sostener la tecnología (Gascó-Hernández, 2018). Esta lección respalda dos decisiones del proyecto: priorizar componentes de código abierto que la municipalidad pueda operar sin depender de un proveedor (ADR-01 y ADR-05) y documentar cada decisión en lenguaje comprensible para la dirección (los ADRs).

Detección acústica de disparos en St. Louis

Mares y Blackburn (2012) evaluaron un sistema de detección acústica de disparos en St. Louis, Estados Unidos. Encontraron que el sistema no redujo de forma significativa la violencia y que, aunque bajaron las llamadas ciudadanas por disparos, el total de despliegues policiales por disparos aumentó de manera importante al sumarse las alertas automáticas. La lección es que un sensor aislado genera muchas alertas que consumen recursos de respuesta. Por eso, en nuestro sistema un disparo detectado no basta por sí solo para una alerta crítica: la regla R02 exige además una probabilidad de al menos 0.7 y un pánico en el mismo sector y en la misma ventana, y el escenario ESC-RUIDO-01 demuestra que un sonido de baja probabilidad termina como falso positivo y no moviliza a ninguna institución.

Tabla 44

Comparación con casos reales

Aspecto	Río de Janeiro	Barcelona	St. Louis	Este proyecto
Enfoque	Centro de operaciones integrado	Estrategia y gobernanza de ciudad inteligente	Sensor acústico aislado	Correlación de varias fuentes con visualización integrada
Escala y costo	Ciudad grande, instalación dedicada	Ciudad grande, programa municipal	Despliegue por zonas	Municipio mediano, un servidor y código abierto
Lección aplicada	Integrar fuentes y avisar a varias agencias	Gobernanza y capacidad propia	Evitar alertas por un solo sensor	Reglas multifuente, análisis de falsos positivos y ADRs

Limitaciones y trabajo futuro

- El servidor único es un punto de falla; se recomienda una UPS, respaldos diarios y, a futuro, un segundo servidor.
- Kafka con un solo broker no replica los datos, con dos servidores o más convendría un clúster de tres brokers.
- La regla R01 es sensible al ruido, conviene exigir que la anomalía de velocidad sea del mismo vehículo que salió.
- La clave de partición por sector desbalancea las particiones y limita la escalabilidad del correlador.
- La confianza de cada sensor debería calibrarse con datos reales de operación.
- Elasticsearch corre sin autenticación por ser laboratorio, en producción debe activarse su seguridad.
- Agregar reglas nuevas es sencillo: basta una función en reglas.py que reciba el evento nuevo y la ventana y devuelva la alerta.

Conclusiones

Se diseñó y construyó una simulación funcional de un sistema de monitoreo perimetral urbano con detección, procesamiento, correlación y visualización de eventos críticos sobre una arquitectura orientada a eventos y distribuida a nivel lógico. La correlación de varias fuentes permitió inferir incidentes que ningún sensor indica por sí solo y distinguir los falsos positivos, y los nueve escenarios de prueba generaron su alerta esperada.

La decisión de desplegar on-premise en un solo servidor resultó adecuada para el cliente: con un costo bajo y los datos bajo control municipal, el sistema sostiene unas 100 veces el tráfico normal cumpliendo el objetivo de latencia, y no pierde datos ante la caída de sus componentes, salvo en una caída prolongada del broker. Las pruebas mostraron además el valor de medir: dos de los hallazgos más importantes, una espera innecesaria de un segundo en el correlador y una lectura de estado que lo hacía diez veces más lento bajo carga, solo aparecieron al medir la latencia y la capacidad.

Por último, los casos de Río de Janeiro, Barcelona y St. Louis confirman que el valor de una ciudad inteligente no está en acumular sensores, sino en integrarlos con criterio, gobernarlos con capacidad propia y evitar que generen más alertas de las que la ciudad puede atender, que fueron los principios que guiaron este proyecto.

Referencias

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media.

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM, 59*(5), 50–57. https://doi.org/10.1145/2890784

C40 Cities. (s.f.). *Rio Operations Center*. https://www.c40.org/case_studies/rio-operations-center

Congreso de la República de Guatemala. (2008). *Ley de Acceso a la Información Pública* (Decreto 57-2008).

Etzion, O., & Niblett, P. (2010). *Event processing in action*. Manning.

Gascó-Hernández, M. (2018). Building a smart city: Lessons from Barcelona. *Communications of the ACM, 61*(4), 50–57. https://cacm.acm.org/magazines/2018/4/226370-building-a-smart-city/fulltext

Instituto Nacional de Estadística. (2019). *XII Censo Nacional de Población y VII de Vivienda 2018*. https://www.censopoblacion.gt

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Krawczyk, H., Bellare, M., & Canetti, R. (1997). *HMAC: Keyed-hashing for message authentication* (RFC 2104). Internet Engineering Task Force. https://doi.org/10.17487/RFC2104

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. En *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*.

Luckham, D. C. (2002). *The power of events: An introduction to complex event processing in distributed enterprise systems*. Addison-Wesley.

Mares, D., & Blackburn, E. (2012). Evaluating the effectiveness of an acoustic gunshot location system in St. Louis, MO. *Policing: A Journal of Policy and Practice, 6*(1), 26–42.

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal, 2014*(239), Artículo 2.

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Parlamento Europeo y Consejo de la Unión Europea. (2016). *Reglamento (UE) 2016/679 relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales*. Diario Oficial de la Unión Europea, L 119.

Singer, N. (2012, 3 de marzo). Mission control, built for cities. *The New York Times*.

Sinnott, R. W. (1984). Virtues of the haversine. *Sky and Telescope, 68*(2), 159.

Anexo A. Registros de decisiones de arquitectura (ADRs)

Este anexo reúne las decisiones de arquitectura del sistema de monitoreo perimetral urbano para la Municipalidad de San Juan Sacatepéquez. Cada decisión se registra como un Architecture Decision Record (ADR), un formato breve propuesto por Nygard (2011) para dejar por escrito el problema que obligó a decidir, las alternativas evaluadas, la elección y sus consecuencias, de modo que cualquier persona pueda entender después por qué el sistema es como es.

Todos los ADRs siguen el formato obligatorio de la sección 4.1 del enunciado del proyecto: contexto, opciones consideradas (mínimo dos alternativas reales con sus ventajas y desventajas), decisión, consecuencias e impacto en el negocio redactado para un directivo no técnico. Además, cuando existe, se agrega la evidencia medida en las pruebas del sistema, para que cada decisión se respalde con números y no solo con argumentos. Los ADR-01 a ADR-08 corresponden a las decisiones obligatorias de la sección 4.2 del enunciado del proyecto; los ADR-09 a ADR-12 son decisiones adicionales del proyecto.

ADR-01. Plataforma de despliegue

Contexto

El sistema debe recibir eventos de sensores, correlacionarlos y avisar a las instituciones en pocos segundos. La municipalidad tiene presupuesto limitado, no cuenta con un equipo de operación en la nube y maneja datos sensibles (placas y ubicaciones de vecinos). El enunciado del proyecto pide elegir entre un modelo on-premise, híbrido o cloud-native, con un costo estimado y la latencia esperada de extremo a extremo.

El volumen normal estimado es de unos 5 eventos por segundo en todo el municipio, con picos de 20 veces ese valor en emergencias o pruebas de carga.

Opciones consideradas

Tabla A1

Opciones de plataforma de despliegue (ADR-01)

Opción	Ventajas	Desventajas
On-premise / LAN (un servidor propio)	Costo fijo y bajo; los datos no salen de la municipalidad; latencia baja y predecible; no depende del internet para recibir y procesar alertas	Sin elasticidad ante picos; un solo servidor es un punto único de falla; la municipalidad mantiene el equipo
Híbrido (ingesta local + analítica en la nube)	Ingesta cercana a los sensores; analítica y respaldos elásticos en la nube	Costo de transferencia de datos (egress); dos entornos que operar e integrar; los datos personales salen del municipio
Cloud-native (servicios gestionados)	Elasticidad; alta disponibilidad gestionada por el proveedor; sin hardware propio	Costo mensual variable y permanente; dependencia del proveedor (vendor lock-in) y del enlace a internet; latencia adicional desde los sensores

Decisión

Se elige **on-premise en un solo servidor**: todo el sistema corre en contenedores dentro de un servidor propio de la municipalidad (en el prototipo, una laptop con Intel i7-13650HX y 32 GB de RAM, de los cuales se asignan 20 GB y 12 núcleos a Docker). Aunque comparten máquina, cada servicio se comunica con los demás solo por red, de modo que la arquitectura sigue siendo distribuida a nivel lógico: si la municipalidad compra un segundo servidor, los contenedores se reparten sin cambiar código.

Consecuencias

Positivas:

- Costo bajo y predecible, sin factura mensual de nube
- Control total de los datos personales dentro de la municipalidad

- Latencia baja y estable, sin depender de la calidad del internet
- Puesta en marcha con un solo comando sobre la configuración versionada

Negativas (aceptadas):

- Sin elasticidad automática: un pico mayor a la capacidad medida se procesa con retraso (no se pierde)
- El servidor es un punto único de falla; se mitiga con UPS, respaldos y un segundo equipo en el futuro
- La municipalidad es responsable del mantenimiento del equipo

Impacto en el negocio

Costo (orden de magnitud, no es cotización): el prototipo usa una laptop ya disponible (unos Q12 000 a Q15 000). Para producción se recomienda un servidor de 8 a 16 núcleos, 32 a 64 GB de RAM y discos en espejo, más una UPS: entre Q28 000 y Q45 000 de inversión única, más unos Q200 mensuales de electricidad. Todo el software es de código abierto, sin licencias. Un equivalente con servicios gestionados en la nube (Kafka, PostgreSQL, Redis, búsqueda, Airflow y cómputo para los contenedores) se estima en US$900 a US$1 300 al mes según precios públicos de lista, es decir, unos Q84 000 a Q120 000 al año: la inversión on-premise se recupera en menos de seis meses.

Riesgo: el principal es la falla del servidor; se acepta para esta escala y se mitiga con respaldos diarios de la base de datos y la opción de agregar un segundo servidor. **Tiempo de implementación:** el sistema completo se levanta en menos de una hora siguiendo el manual (README).

Latencia esperada de extremo a extremo: menos de 2 segundos (objetivo del ADR-08). Medido: entre 30 y 200 ms con tráfico normal y p95 de 976 ms con 508 eventos por segundo sostenidos.

Evidencia medida

- Consumo bajo carga máxima sostenida: unos 4.6 GB de 20 GB de RAM y menos de 1 núcleo de 12
- Capacidad medida: unos 510 eventos/s de extremo a extremo dentro del SLO, unas 100 veces el tráfico normal

ADR-02. Broker de eventos

Contexto

Los sensores producen eventos a ritmos distintos y el correlador, el despachador y otros consumidores deben leerlos sin perder ninguno, en orden por sector y aunque alguno de ellos esté caído por un rato. Se necesita un intermediario (broker) que desacople a quienes producen de quienes consumen.

Opciones consideradas

Tabla A2

Opciones de broker de eventos (ADR-02)

Opción	Ventajas	Desventajas
Apache Kafka	Registro persistente y ordenado por partición; los mensajes quedan guardados aunque ya se hayan leído (permite reprocesar); alto throughput; varios grupos de consumidores leen lo mismo	Más complejo de operar que una cola simple; consume más memoria
RabbitMQ	Simple de operar; enrutamiento flexible; buena latencia	Modelo de cola: el mensaje se borra al consumirse, lo que dificulta reprocesar el historial; menor throughput sostenido
Servicio gestionado (Amazon SQS, Google Pub/Sub)	Sin operación propia; escala solo	Requiere nube (contradice el ADR-01); costo por mensaje; dependencia del internet; latencia mayor

Decisión

Se elige **Apache Kafka 4.3** (Kreps et al., 2011) en modo KRaft, es decir, sin Zookeeper: un solo contenedor que es a la vez broker y controlador. Los topics los crea un contenedor de inicialización (`kafka-init`) y está desactivada la creación automática de topics. La retención es de 7 días y los productores usan `acks=all` con idempotencia.

Consecuencias

Positivas:

- Si el correlador se cae, los eventos esperan guardados en Kafka y se procesan al volver
- El orden por sector se garantiza con la clave de partición (ADR-09)
- El correlador y el despachador leen los mismos datos sin interferirse (grupos de consumidores distintos)
- La ingestión medida supera los 26 500 eventos/s

Negativas (aceptadas):

- Un solo broker significa factor de replicación 1: si se daña el disco, se pierde el historial de Kafka (riesgo aceptado del ADR-01)
- Kafka es el único componente cuya caída prolongada puede perder eventos (ver ADR-06)

Impacto en el negocio

Costo: cero en licencias; unos 450 MB de RAM del servidor. **Riesgo:** bajo para la escala municipal; la caída de Kafka se detecta de inmediato porque los ingestores responden HTTP 503. **Tiempo:** configuración incluida en el despliegue estándar, sin trabajo adicional.

Evidencia medida

- Kafka caído 32 s con tráfico x20: 0 eventos perdidos; caído 70 s: unos 300 perdidos de 13 215 (2.3 %)
- Correlador detenido con 15 000 eventos en cola: Kafka los conservó todos y se procesaron al volver

ADR-03. Persistencia de eventos

Contexto

Los eventos son datos con marca de tiempo y formatos distintos según el sensor. Se necesita guardar eventos, alertas, envíos a instituciones y estadísticas para auditoría y visualización, mantener un estado de corto plazo muy rápido para correlacionar y permitir búsquedas históricas de texto. El volumen normal es de unos 430 000 eventos por día.

Opciones consideradas

Tabla A3

Opciones de persistencia (ADR-03)

Opción	Ventajas	Desventajas
PostgreSQL (relacional) con JSONB	Confiable y conocido; transacciones; uniones entre eventos, alertas y sectores; JSONB guarda formatos distintos sin una tabla por sensor; Grafana lo lee directo	No está optimizado para series de tiempo enormes
Base de series de tiempo (TimescaleDB, InfluxDB)	Muy eficiente para métricas en el tiempo y agregaciones por intervalo	InfluxDB no maneja bien las relaciones entre alertas y eventos; TimescaleDB agrega una extensión más que mantener; el volumen actual no lo necesita
NoSQL (MongoDB, Cassandra)	Esquema flexible; escala horizontal	Consultas de auditoría con uniones más difíciles; consistencia más débil; otro motor que aprender

Decisión

Se usa **PostgreSQL 16** como almacén principal (eventos con `data` y `metadata` en JSONB, alertas, envíos, inválidos, violaciones de SLO y estadísticas, todo con `TIMESTAMPTZ` e índices por tiempo y sector), **Redis 7.4** con persistencia AOF como memoria de corto plazo del correlador y **Elasticsearch 8.19** solo para la búsqueda histórica de alertas (perfil completo).

Consecuencias

Positivas:

- Un solo motor relacional cubre auditoría, trazabilidad (eventId y sourceEventIds) y visualización

- Llaves foráneas: la base rechaza un sector inexistente aunque fallara la validación
- Redis responde en microsegundos para las consultas por evento del correlador
- Elasticsearch permite búsquedas de texto sin cargar la base principal

Negativas (aceptadas):

- Tres motores de datos que respaldar y vigilar
- Con un volumen 100 veces mayor sostenido convendría particionar la tabla de eventos o evaluar TimescaleDB

Impacto en el negocio

Costo: cero en licencias; unos 240 MB (PostgreSQL), 180 MB (Redis) y 1.6 GB (Elasticsearch) de RAM bajo carga. **Riesgo:** bajo; tecnologías maduras y documentadas. **Tiempo:** el esquema se crea solo al iniciar el contenedor.

Evidencia medida

- 448 eventos aceptados = 448 en PostgreSQL tras reiniciar el correlador a mitad de tráfico
- 200 alertas en PostgreSQL = 200 documentos en Elasticsearch al cargar el histórico

ADR-04. Orquestación de tareas periódicas

Estado: aceptada. **Fecha:** octubre de 2026.

Contexto

Hay tareas que no son en tiempo real: borrar datos vencidos (privacidad), resumir alertas por día y calcular la proporción de falsos positivos por sector. El enunciado del proyecto pregunta si se justifica la complejidad operativa de Airflow a esta escala.

Opciones consideradas

Tabla A4

Opciones de orquestación de tareas (ADR-04)

Opción	Ventajas	Desventajas
Apache Airflow (standalone)	Interfaz web con historial, reintentos y ejecución manual; estándar de la industria; pedido por el enunciado del proyecto	Pesado: unos 1.3 GB de RAM para tres tareas SQL; más piezas internas
Scheduler liviano (cron, APScheduler)	Muy liviano (unos 30 MB); trivial de configurar	Sin interfaz ni historial de ejecuciones; reintentos y monitoreo a mano

Decisión

Se usa **Apache Airflow 3.3 en modo standalone** (un contenedor con scheduler, servidor web y procesador de DAGs, metadatos en SQLite) con tres DAGs que ejecutan SQL directo: `retencion_datos` (diario), `reporte_diario` (diario y manual, recalcula sin duplicar) y `analisis_falsos_positivos` (cada hora). Airflow **no** se justifica por carga; se justifica por la trazabilidad de las ejecuciones (quién corrió qué y cuándo), la ejecución manual desde la web durante la demostración y porque deja preparado el crecimiento a más tareas de análisis.

Consecuencias

Positivas:

- Historial visible de cada ejecución y su resultado
- Ejecución manual de los reportes desde la interfaz
- Las tareas pueden crecer sin cambiar de herramienta

Negativas (aceptadas):

- Es el servicio que más memoria usa en reposo (1.33 GB)
- SQLite como metadatos no es apto para producción; en producción iría con PostgreSQL

Impacto en el negocio

Costo: cero en licencias; 1.3 GB de RAM del servidor (holgado dentro de los 20 GB). **Riesgo:** bajo; si Airflow falla, el monitoreo en tiempo real sigue funcionando porque no depende de él. **Tiempo:** los tres DAGs se cargan solos al iniciar.

Evidencia medida

- Las tres ejecuciones manuales terminaron en estado success, con 0 errores de importación
- Solo está activo en el perfil completo (full); el perfil de desarrollo (core) no lo levanta

ADR-05. Contenerización y orquestación

Contexto

El perfil completo tiene 15 servicios que deben levantarse en orden (Kafka y la base de datos antes que quienes los usan), con límites de memoria y que se recuperen solos si fallan.

Opciones consideradas

Tabla A5

Opciones de contenerización (ADR-05)

Opción	Ventajas	Desventajas
Docker Compose	Un archivo describe todo; arranque con un comando; perfiles (core y full); healthchecks y orden de arranque; bajo consumo	Un solo nodo; sin autoescalado ni conmutación por falla entre máquinas
Kubernetes	Autoescalado, despliegues sin corte, conmutación por falla entre nodos (Burns et al., 2016)	Plano de control propio (más memoria y complejidad); muchos conceptos nuevos; en un solo servidor no da alta disponibilidad real

Decisión

Se usa **Docker Compose** (Merkel, 2014) con dos perfiles: `core` para desarrollo y `full` para la demostración (agrega Elasticsearch y Airflow). Se decide conscientemente **no usar Kubernetes**: en un solo servidor agrega complejidad sin ganar disponibilidad, porque si la máquina se apaga, todo se cae igual. Cada servicio tiene límite de memoria, healthcheck cuando aplica, dependencias por estado sano y reinicio automático.

Consecuencias

Positivas:

- Instalación reproducible: verificada clonando el repositorio en una carpeta nueva y levantando todo solo con el manual
- Imágenes con versiones fijas (sin latest): el sistema no cambia solo
- Si un servicio falla, Docker lo reinicia

Negativas (aceptadas):

- Sin autoescalado: la capacidad está fijada por el servidor
- Migrar a varios servidores requeriría Docker Swarm o Kubernetes

Impacto en el negocio

Costo: cero; Docker Desktop y Docker Engine son gratuitos para este uso. **Riesgo:** bajo; la configuración está versionada en git. **Tiempo:** instalación de cero a sistema funcionando en menos de una hora.

Evidencia medida

- Perfil completo bajo carga: unos 4.6 GB en total; los servicios Python usan menos de 60 MB cada uno
- Suma de los límites de memoria: unos 12.4 GB, dentro de los 20 GB asignados

ADR-06. Resiliencia y tolerancia a fallos

Contexto

Cualquier componente puede caerse durante la operación o la demostración: un ingestor, el correlador, Redis o el propio broker.

Opciones consideradas

Tabla A6

Opciones de manejo de fallos (ADR-06)

Opción	Ventajas	Desventajas
Entrega al menos una vez + deduplicación	No se pierde nada ante caídas de consumidores; duplicados controlados por eventId	Puede haber trabajo repetido que hay que filtrar
Entrega como máximo una vez	Simple; nunca hay duplicados	Una caída pierde los eventos en proceso: inaceptable para alertas de seguridad
Exactamente una vez (transacciones de Kafka)	Sin duplicados ni pérdidas dentro de Kafka	Mucho más complejo; no cubre escrituras externas (PostgreSQL, Redis, REST) sin coordinación adicional

Decisión

Se adopta **entrega al menos una vez con deduplicación**: el correlador confirma el offset de Kafka solo después de guardar en PostgreSQL, publicar las alertas y marcar los eventos en Redis; si se cae en medio, reprocesa el lote y descarta lo ya hecho por eventId. Además: NGINX saca de servicio un ingestor que falla y reintenta en otro; los eventos inválidos van al topic `events.invalid` (dead-letter) con su motivo; los ingestores responden HTTP 503 si Kafka no confirma en 5 s y el cliente reintenta con espera creciente (1, 2, 4 y 8 s, hasta 5 intentos); el correlador se apaga en orden ante SIGTERM y Docker reinicia cualquier servicio caído. **Pérdida aceptable:** cero ante la caída de cualquier componente distinto de Kafka; ante una caída de Kafka, el cliente aguanta unos 40 s por lote y más allá la pérdida es proporcional a la duración de la caída.

Consecuencias

Positivas:

- Medido: 0 eventos perdidos y 0 alertas duplicadas al caer un ingestor, el correlador o Redis

- Ningún evento inválido se descarta en silencio

Negativas (aceptadas):

- Una caída de Kafka mayor a unos 40 s pierde eventos (2.3 % en una prueba de 70 s)
- Para eliminar esa pérdida, los sensores necesitarían guardar y reenviar (store and forward)

Impacto en el negocio

Costo: ninguno adicional; son decisiones de diseño. **Riesgo:** acotado y medido; el único escenario con pérdida es una caída prolongada del broker, que se detecta de inmediato. **Tiempo:** incluido en el desarrollo.

Evidencia medida

- Caída de ingestor-2 por 20 s: 9003 aceptados = 9003 guardados
- Reinicio del correlador: de vuelta en menos de 1 s gracias al apagado ordenado
- Kafka caído 70 s: 251 eventos que el cliente dio por perdidos sí llegaron (quedaron en la cola del productor) y la deduplicación evitó contarlos dos veces

ADR-07. Seguridad y privacidad de datos

Contexto

Las placas vehiculares y la ubicación geográfica son datos personales. Aunque en la simulación son sintéticos, el enunciado del proyecto exige tratarlos como si fueran reales. En Guatemala, la Ley de Acceso a la Información Pública (Congreso de la República de Guatemala, 2008) reconoce la protección de los datos personales en poder de las instituciones públicas, y el Reglamento General de Protección de Datos europeo (Parlamento Europeo y Consejo de la Unión Europea, 2016) es una referencia internacional para la seudonimización.

Opciones consideradas

Tabla A7

Opciones de protección de placas (ADR-07)

Opción	Ventajas	Desventajas
Guardar la placa en claro con control de acceso	Simple; permite ver la placa completa	Cualquier fuga expone datos personales; contradice el principio de minimización
Cifrado reversible de la placa	Se puede recuperar la placa con la clave	La clave debe estar disponible en todos los servicios: una fuga de la clave expone todo
Seudonimización con HMAC-SHA256	Sin la clave no se puede recuperar la placa; dos lecturas de la misma placa dan el mismo hash (permite correlacionar); el operador ve solo una parte	No se puede recuperar la placa completa desde el sistema (se requiere un proceso aparte con la autoridad)

Decisión

Se adopta **seudonimización con HMAC-SHA256** (Krawczyk et al., 1997): la cámara envía solo el hash con la clave `PLATE_HMAC_KEY` y una versión parcial (por ejemplo `P-***BCD`). El ingestor rechaza con el motivo `PLACA_EN_CLARO` cualquier evento que traiga la placa completa. Además: retención de 30 días para eventos y 365 para alertas (DAG `retencion_datos`); tres usuarios de base de datos con mínimo privilegio (`app`, `grafana_ro` solo lectura y `admin` solo para la creación); contraseñas solo en el archivo `.env`, que no se sube al repositorio; bases de datos publicadas únicamente en 127.0.0.1; y el identificador de usuario de la app es un seudónimo. Elasticsearch corre sin autenticación **solo** por ser laboratorio en una máquina; en producción se activaría su seguridad.

Consecuencias

Positivas:

- Una fuga de la base de datos no revela placas ni identidades

- La correlación de vehículos (R05 y R07) funciona igual con hashes
- Grafana no puede modificar datos aunque alguien edite una consulta

Negativas (aceptadas):

- Recuperar una placa completa para una investigación requiere un proceso fuera del sistema
- Elasticsearch sin autenticación es aceptable solo en laboratorio

Impacto en el negocio

Costo: ninguno. **Riesgo:** reduce el riesgo legal y reputacional de la municipalidad ante una fuga de datos. **Tiempo:** incluido en el desarrollo.

Evidencia medida

- Validado: un evento con placa en claro es rechazado con PLACA_EN_CLARO y queda registrado en el dead-letter
- El usuario grafana_ro recibe permission denied al intentar escribir

ADR-08. Observabilidad y SLOs

Contexto

No basta con tener dashboards: hay que definir umbrales concretos de servicio y qué ocurre cuando se incumplen (Beyer et al., 2016). Una alerta de robo en curso pierde valor si llega tarde, y un porcentaje alto de eventos inválidos indica sensores dañados o mal configurados.

Opciones consideradas

Tabla A8

Opciones de observabilidad (ADR-08)

Opción	Ventajas	Desventajas
SLOs explícitos medidos en el propio flujo	Cada alerta registra su latencia; los incumplimientos quedan en una tabla y en el log; visible en Grafana	Requiere diseñar qué medir desde el inicio
Solo dashboards de métricas generales	Rápido de armar	No define cuándo algo está mal ni deja evidencia de los incumplimientos

Decisión

Se definen dos SLOs: **(1) latencia de alertas p95 menor a 2 000 ms**, medida como la hora de creación de la alerta menos el `emittedAt` del último evento que completó la regla; **(2) menos de 2 % de eventos inválidos.** Cuando una alerta supera el umbral, el correlador escribe una advertencia en el log y la registra en la tabla `slo_violations`; el dashboard 2 de Grafana muestra el p95 en rojo, la línea del umbral, el porcentaje de inválidos y la lista de violaciones. Los falsos positivos no cuentan para el SLO de latencia porque, por definición, esperan la ventana completa.

Consecuencias

Positivas:

- El incumplimiento queda registrado y visible, no solo se nota
- Permitió encontrar y corregir problemas reales (una espera de 1 s en el correlador y el reparto lento de particiones tras un reinicio)

Negativas (aceptadas):

- Al procesar un atraso tras una caída se registran muchas violaciones; es correcto, porque reflejan lo que vivió el operador

Impacto en el negocio

Costo: ninguno adicional. **Riesgo:** reduce el riesgo de operar a ciegas: la dirección puede ver si el servicio cumple lo prometido. **Tiempo:** incluido en el desarrollo.

Evidencia medida

- Latencia de alertas con tráfico normal: de 30 a 200 ms
- A 508 eventos/s sostenidos: p95 de 976 ms (cumple); a 661 eventos/s: p95 de 13.7 s (incumple, sin pérdida)
- Eventos inválidos: entre 1.4 y 1.5 % (cumple)

ADR-09. Diseño de topics y clave de partición

Contexto

Hay que decidir cómo organizar los mensajes en Kafka. El correlador necesita ver juntos y en orden los eventos de distintos tipos de un mismo sector (por ejemplo, pánico, luego cámara y luego radar en Ciudad Quetzal) para aplicar las reglas.

Opciones consideradas

Tabla A9

Opciones de diseño de topics (ADR-09)

Opción	Ventajas	Desventajas
Un topic por tipo de sensor	Cada tipo tiene su canal; un consumidor puede leer solo el tipo que le interesa	El correlador tendría que leer cinco topics y reordenar los eventos entre ellos: se pierde el orden entre tipos de un mismo sector, que es justo lo que la correlación necesita
Un topic por etapa del flujo, con clave = sector	Todos los eventos de un sector llegan en orden a un solo consumidor; cada etapa (eventos válidos, inválidos, alertas, avisos externos) tiene su propio topic y sus consumidores	Las particiones quedan desbalanceadas porque los sectores tienen tráfico distinto

Decisión

Se usan **seis topics por etapa del flujo**: `events.standardized` (3 particiones, clave = sector), `events.invalid` (dead-letter), `correlated.alerts` (3 particiones) y `external.pnc`, `external.bomberos` y `external.conred` (uno por institución). Separar por etapa sí organiza a los consumidores: el correlador solo lee eventos válidos, el despachador solo lee alertas e inválidos y cada institución solo lee su propio topic.

Consecuencias

Positivas:

- Orden garantizado por sector para la correlación
- Cada institución recibe únicamente lo que le corresponde

Negativas (aceptadas):

- Desbalance medido: tras 2 minutos de tráfico las particiones tenían 163, 491 y 71 mensajes
- Limita la escalabilidad horizontal del correlador (ver ADR-10)

Impacto en el negocio

Costo: ninguno. **Riesgo:** bajo; el orden por sector evita alertas mal formadas. **Tiempo:** incluido en la configuración inicial.

ADR-10. Modelo de correlación e inferencia

Contexto

El sistema debe inferir situaciones como "posible robo en curso" o "accidente vial" a partir de eventos de varias fuentes, y distinguir los falsos positivos. La lógica debe ser explicable ante actores no técnicos (dirección, área legal), porque una alerta puede movilizar a la PNC.

Opciones consideradas

Tabla A10

Opciones de modelo de correlación (ADR-10)

Opción	Ventajas	Desventajas
Reglas determinísticas con ventana de tiempo (estado en Redis)	Explicables y auditables (cada alerta indica su regla y sus eventos de origen); fáciles de probar con escenarios; sin datos de entrenamiento	Las reglas se mantienen a mano; solo detectan patrones previstos
Motor de procesamiento de eventos complejos (Apache Flink, Esper)	Lenguaje especializado para patrones y ventanas (Luckham, 2002; Etzion & Niblett, 2010); escala a volúmenes grandes	Otra plataforma pesada que operar; curva de aprendizaje alta para el equipo
Aprendizaje automático	Puede descubrir patrones no previstos	Requiere datos históricos reales etiquetados que no existen; difícil de explicar ante el área legal

Decisión

Se implementan **siete reglas determinísticas (R01 a R07) más la regla de falso positivo**, evaluadas por un correlador en Python que mantiene en Redis una ventana de tiempo por sector y tipo de evento (300 s en operación, 60 s en la demostración). Cada alerta lleva un score de 0 a 1 calculado con la confianza de los sensores, la lista de eventos de origen y la regla que la generó. El detalle está en el capítulo Modelo de correlación e inferencia.

Consecuencias

Positivas:

- Cada alerta es auditable hasta el sensor que la originó
- Probado con 9 escenarios guionados: 9 de 9 alertas esperadas
- Escala horizontalmente porque todo el estado está en Redis

Negativas (aceptadas):

- La regla R01 tal como está definida es sensible al ruido: en tráfico normal casi siempre hay un vehículo saliendo, por lo que en la práctica basta pánico más exceso de velocidad; una mejora futura es exigir que la anomalía sea del mismo vehículo
- Una ventana mayor aumenta las correlaciones por azar

Impacto en el negocio

Costo: ninguno adicional. **Riesgo:** la explicabilidad reduce el riesgo de enviar patrullas por una decisión que nadie puede justificar. **Tiempo:** agregar una regla nueva es una función corta en un solo archivo.

Evidencia medida

- Capacidad del correlador: 660 eventos/s con una instancia y 743 con tres (limitado por la partición más cargada)
- Optimización medida: la primera versión procesaba 66 eventos/s; la versión final, 660

ADR-11. Lenguaje de implementación

Contexto

El diagrama de referencia tomado fue el explicado en clase donde se muestra ingestores en Java y Node.

Opciones consideradas

Tabla A11

Opciones de lenguaje (ADR-11)

Opción	Ventajas	Desventajas
Varios lenguajes (Java, Node, Python)	Muestra heterogeneidad tecnológica como en el diagrama	Tres ecosistemas que aprender, compilar y mantener.
Un solo lenguaje (Python)	Código legible; librerías maduras (FastAPI, Pydantic, confluent-kafka);	Rendimiento por proceso menor que Java; el correlador es el cuello de botella medido

Decisión

Se usa **Python 3.12 en todos los servicios** (ingestores, correlador, despachador, aseguradora simulada, simulador y DAGs). La heterogeneidad que pide el enunciado del proyecto está en los datos (cinco formatos de sensor), no en los lenguajes.

Consecuencias

Positivas:

- Cada servicio es autocontenido, con versiones fijas en su requirements.txt

Negativas (aceptadas):

- El correlador en un solo proceso Python limita el máximo de extremo a extremo a unos 510 eventos/s; suficiente para el municipio

Impacto en el negocio

Costo: ninguno. **Riesgo:** reduce la dependencia de una sola persona que conozca un lenguaje. **Tiempo:** menor tiempo de desarrollo y de capacitación.

ADR-12. Formato de eventos

Contexto

Cinco tipos de sensores envían datos con estructuras distintas. Se necesita un contrato común que sea fácil de validar, de leer en las pantallas de depuración y de evolucionar en el futuro.

Opciones consideradas

Tabla A12

Opciones de formato de eventos (ADR-12)

Opción	Ventajas	Desventajas
JSON con schemaVersion y validación en el ingestor	Legible a simple vista (Kafka UI, logs); sin servidores adicionales; Pydantic valida tipos y catálogos con poco código	Mensajes más grandes que un formato binario; la compatibilidad se controla a mano
Avro con Schema Registry	Formato binario compacto; compatibilidad de esquemas verificada automáticamente	Un servidor más que operar; los mensajes no se leen a simple vista.

Decisión

Se usa **JSON con el campo schemaVersion,** validado con Pydantic en el ingestor. Los inválidos se rechazan con un motivo legible (por ejemplo, `ZONA_DESCONOCIDA` o `TIMESTAMP_EN_EL_FUTURO`).

Consecuencias

Positivas:

- Cualquier persona puede leer un evento en Kafka UI
- El campo schemaVersion permite introducir una versión 2.0 sin romper a los consumidores actuales

Negativas (aceptadas):

- Más bytes por mensaje que Avro; irrelevante al volumen actual

Impacto en el negocio

Costo: ninguno. **Riesgo:** bajo. **Tiempo:** menor que integrar un registro de esquemas.